V-SPACE FPGA Build Challenge 2026

Final Report

Five FPGA experiments on the AMD PYNQ-Z2 (Zynq-7000 XC7Z020)

Team

Team name	Semicons
Team leader	K S Sanjay Sri Ramnath
Registration number	24BEC0209

Board and tools

AMD (TUL) PYNQ-Z2 with the Zynq-7000 XC7Z020-1CLG400C: a dual-core ARM Cortex-A9 processing system and programmable logic with 53,200 LUTs, 106,400 flip-flops, 140 block RAMs and 220 DSP slices. The designs are written in SystemVerilog, simulated with Icarus Verilog 12 and built with Vivado 2026.1. In every experiment the hardware sits in the FPGA fabric and the ARM processor controls it over an AXI4-Lite register interface at 0x4000_0000, from a Python (PYNQ) notebook.

Summary and contents

#	Level	Experiment	Result on the board	Page
1	Beginner	**Hardware True Random Number Generator**	100,000 bytes: 50.02 % ones, entropy 7.9982 bits per byte, NIST frequency and runs tests pass	2
2	Beginner	**Huffman Compression Accelerator**	600-byte text compressed to 57.6 % of its size and decoded back exactly	10
3	Intermediate	**Pipelined AES-256 Encryption Engine**	FIPS-197 test vector correct; a 48-byte message encrypted and decrypted correctly	18
4	Intermediate	**NeuroCore-32 RISC-V Processor**	Fibonacci and bubble sort programs correct; 0.955 instructions per clock	26
5	Advanced	**Hardware Q-Learning Agent**	2000 episodes in 4.9 ms; 14-step shortest path found, same result as simulation	35

Every design meets timing and passes its simulation testbench, and all five were run on the board on 2026-09-18. Each report covers the objective, block diagram, RTL design, simulation results, hardware implementation, applications and conclusion. The source code, testbenches, simulation files and images are in the Experiment-N folders of the repository.

Experiment 1: Hardware True Random Number Generator

FPGA Build Challenge 2026 - Beginner level - AMD PYNQ-Z2 (Zynq-7000 XC7Z020)

1. Objective

Computers normally make "random" numbers with a formula (a pseudo-random generator). If someone knows the formula and its starting value, every number can be predicted, which is not acceptable for things like encryption keys. The objective of this experiment is to build a **true** random number generator (TRNG) in the FPGA fabric that gets its randomness from a physical source - the timing jitter of free-running ring oscillators - and to deliver the random bits to the ARM processor over AXI4-Lite. The output is then checked with the NIST frequency and runs tests.

2. Block Diagram

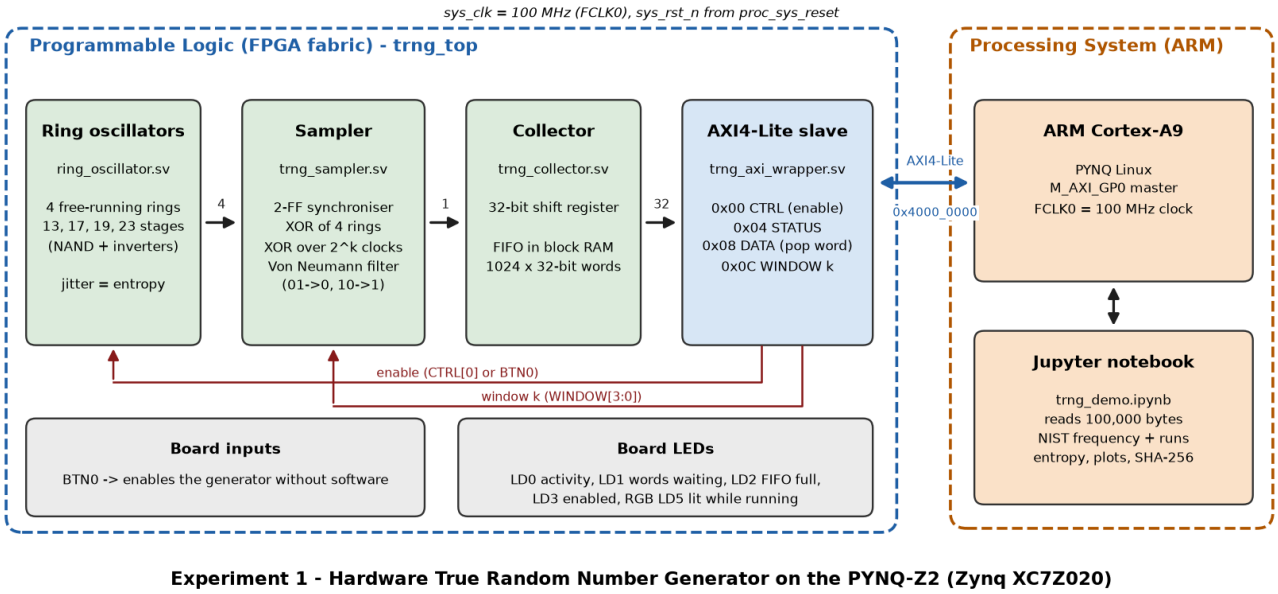


Figure 1 - System block diagram

Four ring oscillators of different lengths run freely inside the FPGA. Their outputs are brought into the 100 MHz clock domain and XORed together. Because samples one clock apart are still correlated, the sampler XORs 2^k samples into one bit (k is set by software, default 6). A Von Neumann filter removes any bias. The collector packs the bits into 32-bit words and stores them in a 1024-word FIFO in block RAM. The ARM processor reads the words through four AXI4-Lite registers, and a Jupyter notebook analyses them.

3. RTL Design

3.1 Module hierarchy

```
system_wrapper                (Vivado block design)
 |-- processing_system7_0     ARM Cortex-A9, gives FCLK0 = 100 MHz and M_AXI_GP0
 |-- rst_ps7_0_100M           reset synchroniser
 |-- ps7_0_axi_periph         AXI interconnect / protocol converter
 +-- trng_top_0               trng_top.sv
      |-- ro_inst             ring_oscillator.sv  (4 x ro_core)
      |-- sampler_inst        trng_sampler.sv
      |-- collector_inst      trng_collector.sv
      +-- axi_inst            trng_axi_wrapper.sv
```

3.2 Modules

Module (file)	What it does
ring_oscillator (ring_oscillator.sv)	Four rings of 13, 17, 19 and 23 stages. Each ring is one NAND gate (used as the enable) followed by inverters in a loop. An odd number of inversions makes the loop oscillate on its own. The keep / dont_touch attributes stop Vivado from removing the loop.
trng_sampler (trng_sampler.sv)	Two flip-flops synchronise the oscillator outputs to the clock. The four bits are XORed into one raw bit per clock. A counter XORs 2^k raw bits into one window bit. A Von Neumann filter takes window bits in pairs: 01 gives 0, 10 gives 1, 00 and 11 are thrown away.
trng_collector (trng_collector.sv)	A 32-bit shift register collects the random bits. Every 32 bits the word is written into a 1024 x 32 FIFO built in block RAM. 11-bit read and write pointers (one extra bit) tell full from empty.
trng_axi_wrapper (trng_axi_wrapper.sv)	AXI4-Lite slave with four registers (table below). Reading the DATA register pops one word from the FIFO. A new read address is only accepted after the previous data has been taken.
trng_top (trng_top.sv)	Connects the four blocks. BTN0 (through a 2-FF synchroniser) can also enable the generator. Drives the LEDs: LD0 blinks with activity, LD1 = words waiting, LD2 = FIFO full, LD3 = enabled, RGB LED LD5 lit while running.

3.3 Register map (base address 0x4000_0000)

Offset	Name	Access	Meaning
0x00	CTRL	R/W	bit 0 = enable the generator
0x04	STATUS	R	bit 0 = FIFO empty, bit 1 = FIFO full, bits 31:16 = words waiting
0x08	DATA	R	next random word; reading removes it from the FIFO
0x0C	WINDOW	R/W	bits 3:0 = k, one raw bit every 2^k clock cycles (reset value 6)

3.4 RTL schematic

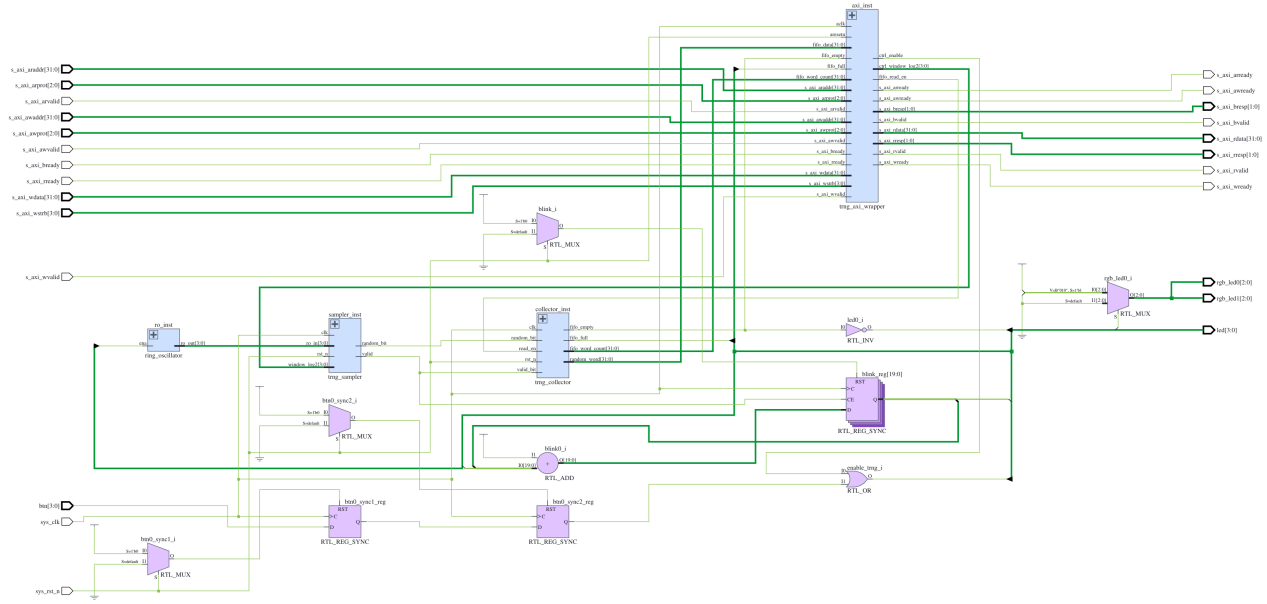


Figure 2 - Elaborated RTL schematic of trng_top (Vivado)

4. Simulation Results

The design was simulated with Icarus Verilog using the testbench *trng_tb.sv*. The testbench checks the register reset value, a register write, an empty-FIFO read, enables the generator, waits 100 us and then reads the status and two random words. All checks passed:

```
status = 004e0000 (78 words waiting)
random words: 3a83365a cb930aca
PASS: TRNG produced random words over AXI
```

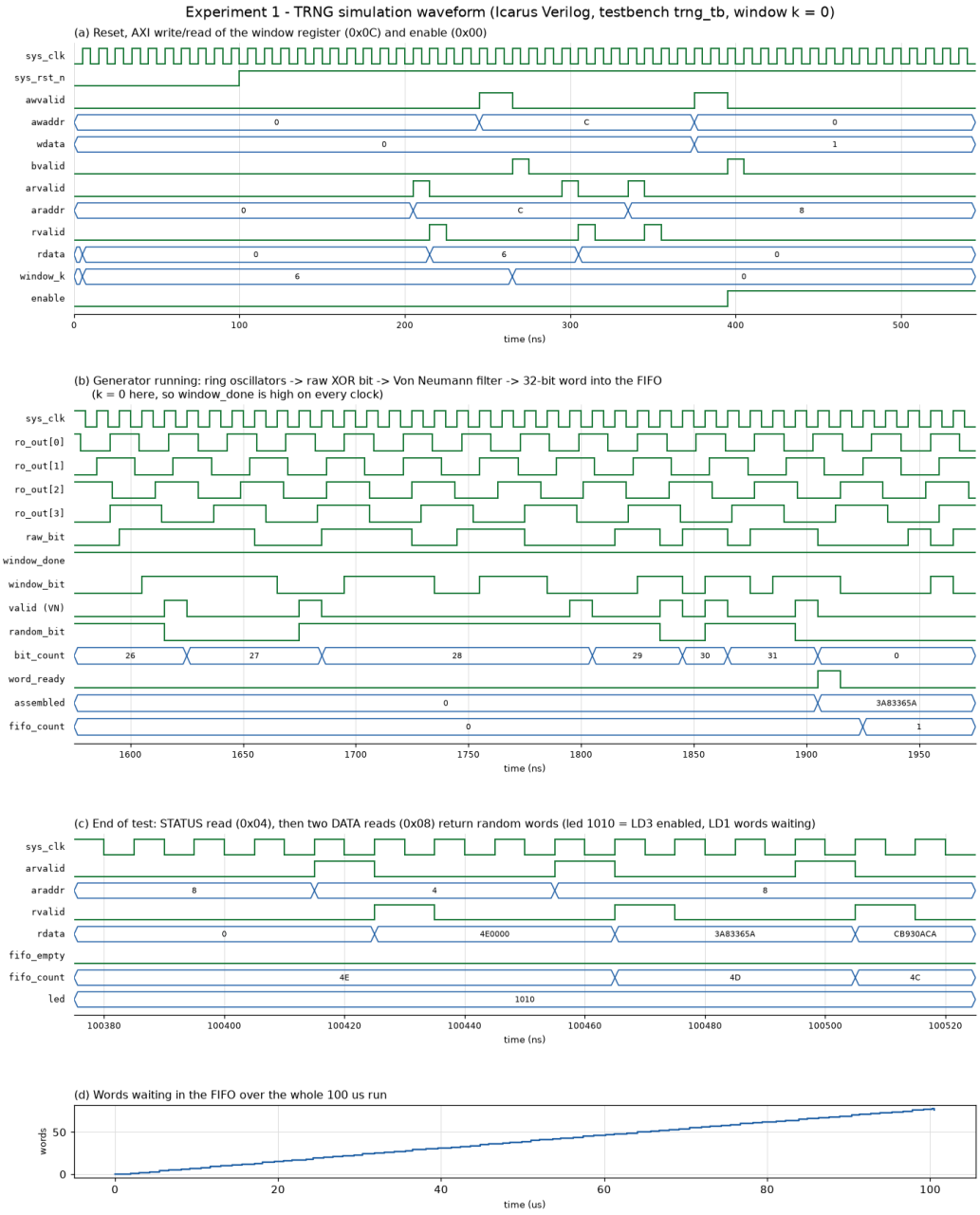


Figure 3 - Simulation waveform (details in Simulation/simulation_report.pdf)

5. Hardware Implementation

5.1 Build

The design was built with Vivado 2026.1 for the xc7z020clg400-1 device using build.tcl. The block design connects the Zynq processing system, a reset block, an AXI interconnect and trng_top. The constraints file (pynq_z2.xdc) assigns the LED and button pins, sets DONT_TOUCH on the ring oscillator cells and ALLOW_COMBINATORIAL_LOOPS on their nets so that the loops pass the design rule check.

Resource	Used	Available	Utilisation
Slice LUTs	571	53,200	1.07 %
Slice registers (flip-flops)	669	106,400	0.63 %
Block RAM tiles (RAMB36)	1	140	0.71 %
Bonded I/O pins	11	125	8.80 %
Global clock buffers	1	32	3.13 %

Timing (100 MHz clock)	Value	Status
Worst setup slack (WNS)	+0.530 ns	**met**
Worst hold slack (WHS)	+0.033 ns	**met**
Failing endpoints	0 of 1656	**met**

5.2 Test on the board

The bitstream was loaded into the PYNQ-Z2, the window was set to k = 6 and the generator was enabled. The 1024-word FIFO filled within 100 ms (LD2 on). 25,000 words (100,000 bytes) were then read from the DATA register and tested with the same code as the Jupyter notebook trng_demo.ipynb.

Test	Result
Bytes tested	100,000
Ones	50.07 % (ideal 50 %)
Shannon entropy	7.9979 / 8 bits per byte
Longest run	20 identical bits
Max |autocorrelation|	0.0062 (lags 1-32)
NIST frequency test	**PASS (p = 0.2386)**
NIST runs test	**PASS (p = 0.2600)**
Repeated 32-bit words	0 of 25,000

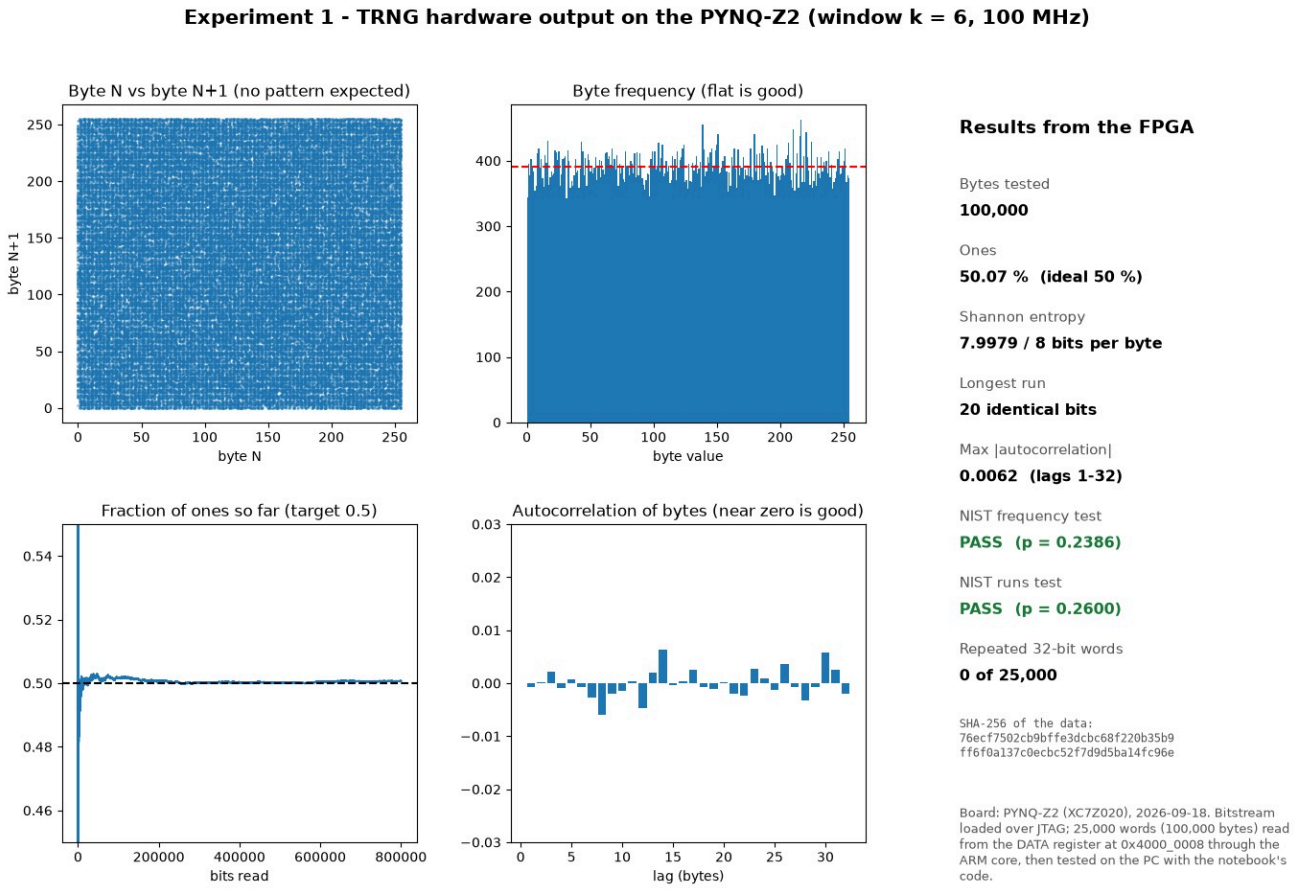


Figure 4 - Random data read from the FPGA and its test results

Run of the Jupyter notebook on the board

The steps of the notebook trng_demo.ipynb (Jupyter folder) were also run on the PYNQ-Z2 with PYNQ, using the same bitstream. tools/execute_all_on_pynq.py runs them on the board over SSH and copies the log and the graph back, and tools/embed_nb_outputs.py puts them into the notebook. It read 100,000 bytes with window k = 6 in 2.0 s and ran the same tests. This is a new set of random bytes, so the numbers differ slightly from the run above, but every test passes again: 50.02 % ones, entropy 7.9982 bits per byte, longest run 17 bits, NIST frequency p = 0.7628 and runs p = 0.5506. The log and the graph the notebook drew are in Jupyter/output.

```
[*] Flashing FPGA bitstream for Experiment 1 (TRNG)...
[+] FPGA Flashed! DONE LED active.
Overlay loaded, TRNG at 0x40000000
Read 100000 bytes in 2.0 s with window k = 6
Ones:           50.02 %
Entropy:        7.9982 / 8.0 bits per byte
Longest run:    17 identical bits
NIST frequency: PASS (p = 0.7628)
NIST runs:      PASS (p = 0.5506)
SHA-256 of the 100,000 bytes: 84aff8366f9fb517edf6d34706c0e748c579ebaee232a52a68a3b44718302f30
```

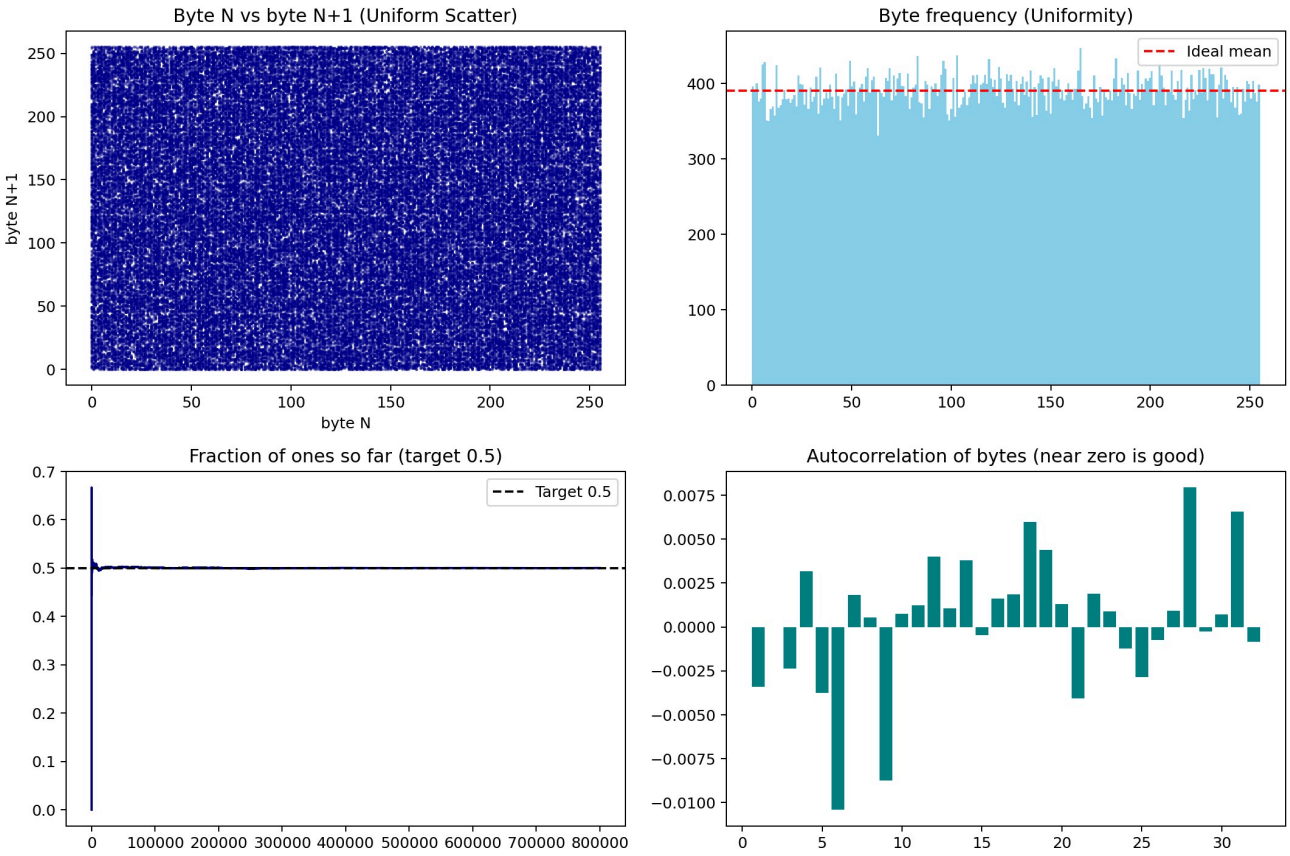


Figure 5 - Graph drawn by trng_demo.ipynb on the board

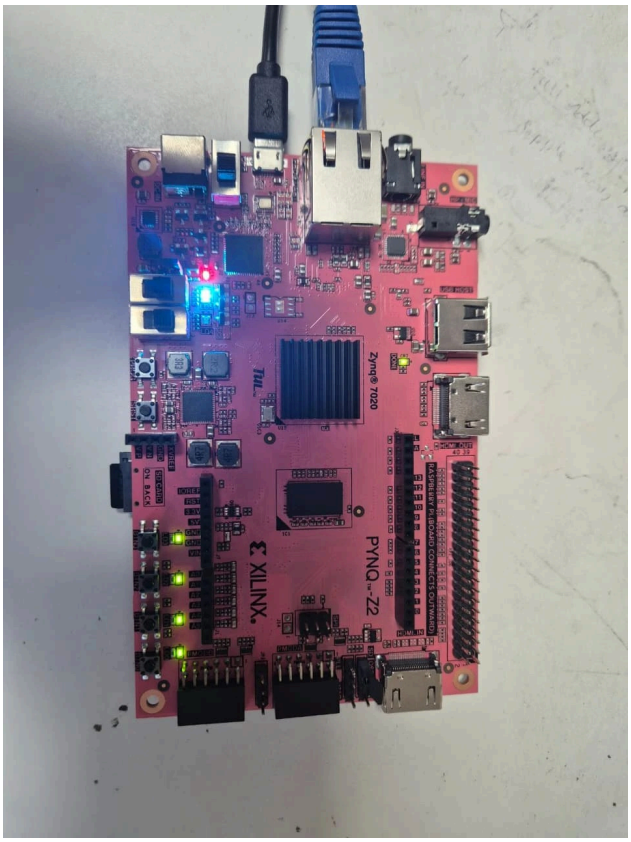


Figure 6 - PYNQ-Z2 board setup: micro-USB (power and JTAG/UART), Ethernet to the PC, PYNQ Linux on the SD card

All plots look as expected for random data: the byte-pair scatter shows no pattern, the byte histogram is flat, the fraction of ones settles at 0.5 and the autocorrelation stays close to zero. Both NIST tests pass (p-value above 0.01).

6. Applications

- **Cryptography:** seeds for encryption keys, nonces and initialisation vectors (after hashing the raw bits, for example with SHA-256).
- **IoT and embedded security:** small devices without a good entropy source can get random numbers from their own FPGA fabric.
- **Authentication:** random challenges for challenge-response login and one-time passwords.
- **Simulation and gaming:** Monte Carlo simulations, lotteries and games that need fair, unpredictable numbers.

7. Conclusion

A working TRNG was designed, simulated and implemented on the PYNQ-Z2. It uses only 1 % of the FPGA's LUTs and one block RAM, meets timing at 100 MHz, and its output passes the NIST frequency and runs tests with 7.998 bits of entropy per byte.

Learning outcomes: how ring oscillators turn electrical noise into random bits and why a synchroniser is needed when sampling them; how a Von Neumann filter removes bias; building a FIFO in block RAM; writing an AXI4-Lite slave and connecting custom logic to the Zynq ARM processor; and testing randomness with statistical tests.

Future scope: add the online health tests from NIST SP 800-90B so that a failed oscillator is detected automatically; add a SHA-256 conditioning block in hardware; stream the data with AXI DMA for higher throughput; and place the oscillators with location constraints so that the entropy is the same after every rebuild.

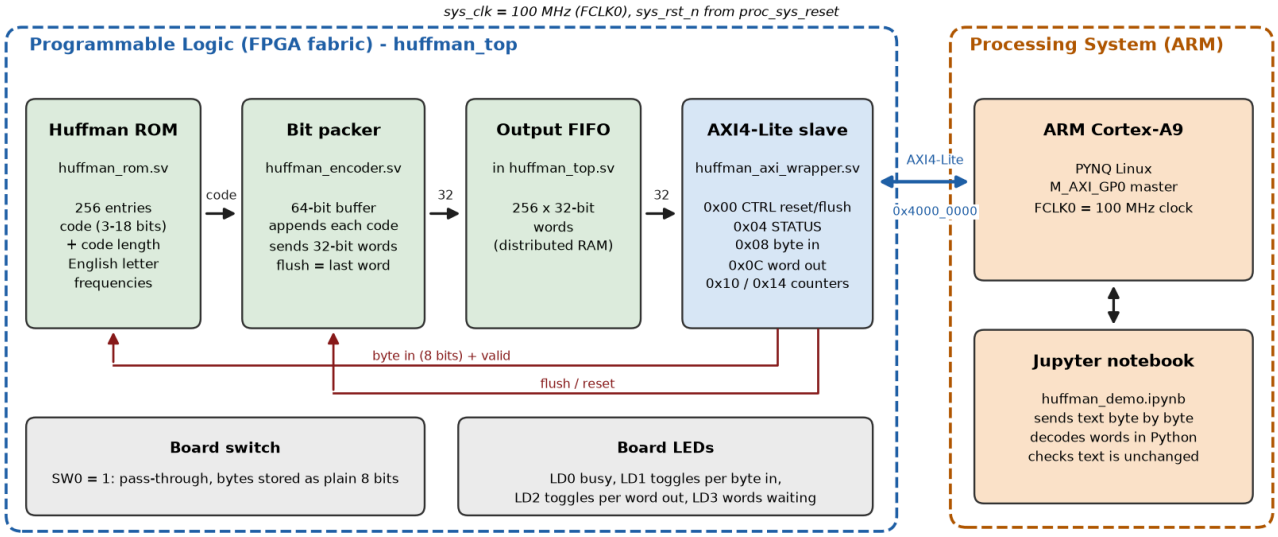
Experiment 2: Huffman Compression Accelerator

FPGA Build Challenge 2026 - Beginner level - AMD PYNQ-Z2 (Zynq-7000 XC7Z020)

1. Objective

Plain text stores every character in 8 bits, although some letters are used far more often than others. Huffman coding gives short codes to common characters and long codes to rare ones, so the same text needs fewer bits and can still be decoded exactly (lossless compression). The objective of this experiment is to build a Huffman encoder in the FPGA that the ARM processor can use over AXI4-Lite: the processor writes text one byte at a time, the hardware looks up each code, packs the variable-length codes into 32-bit words and returns them. The result is checked by decoding it again in software.

2. Block Diagram



Experiment 2 - Huffman Compression Accelerator on the PYNQ-Z2 (Zynq XC7Z020)

Figure 1 - System block diagram

The processor writes a byte to the AXI4-Lite slave. The byte addresses the Huffman ROM, which returns the code (3 to 18 bits) and its length one clock later. The bit packer appends the code to a 64-bit buffer; whenever 32 or more bits are waiting, the lowest 32 bits are sent to a 256-word output FIFO. A flush command sends the last partial word. The processor reads the words and two counters (bytes in, bits out) back through the same slave. Switch SW0 selects a pass-through mode in which each byte is stored as its plain 8 bits.

3. RTL Design

3.1 Module hierarchy

```
system_wrapper               (Vivado block design)
 |-- processing_system7_0    ARM Cortex-A9, gives FCLK0 = 100 MHz and M_AXI_GP0
 |-- rst_ps7_0_100M          reset synchroniser
 |-- ps7_0_axi_periph        AXI interconnect / protocol converter
 +-- huffman_top_0           huffman_top.sv  (output FIFO, counters, LEDs)
      |-- axi_wrapper        huffman_axi_wrapper.sv
      +-- encoder            huffman_encoder.sv
           +-- rom_inst      huffman_rom.sv  (generated by generate_huffman_table.py)
```

3.2 Modules

Module (file)	What it does
generate_huffman_table.py	Python script that builds the code table. It gives each byte a weight from English letter frequencies (every other byte weight 1), runs Huffman's algorithm (repeatedly merge the two lightest nodes) to get a code length per byte, then hands out canonical codes so no code is the start of another. It writes the ROM, the testbench table and the JSON table used by the decoder, so all three always agree.
huffman_rom (huffman_rom.sv)	A 256-entry case statement: byte in, code and code length out, registered (one clock). The codes are stored bit-reversed so that the first code bit ends up in the lowest free bit of the buffer.
huffman_encoder (huffman_encoder.sv)	Delays the byte valid signal by one clock to match the ROM. A multiplexer picks the ROM code or, in pass-through mode (SW0), the raw byte with length 8. The code is shifted left by bit_count and ORed into a 64-bit buffer. If the new count reaches 32 the low 32 bits leave as word_out and the buffer shifts down by 32. A flush sends any remaining bits as a last word and pulses done.
huffman_axi_wrapper (huffman_axi_wrapper.sv)	AXI4-Lite slave with six registers (table below). Writes to CTRL and DATA IN become one-clock pulses. The done pulse is latched in STATUS until the next command so software cannot miss it. Reading DATA OUT pops one word from the FIFO.
huffman_top (huffman_top.sv)	Connects the blocks. Holds the 256 x 32-bit output FIFO (distributed RAM, with a word count), the bytes-in and bits-out counters, and drives the LEDs: LD0 busy, LD1 toggles per byte in, LD2 toggles per word out, LD3 lit while words are waiting.

3.3 Register map (base address 0x4000_0000)

Offset	Name	Access	Meaning
0x00	CTRL	W	bit 0 = reset the engine and counters, bit 1 = flush the last bits
0x04	STATUS	R	bit 0 busy, bit 1 done, bit 2 FIFO empty, bit 3 FIFO full, bits 31:16 words waiting
0x08	DATA IN	W	one byte to compress
0x0C	DATA OUT	R	next compressed word; reading removes it from the FIFO
0x10	BYTES IN	R	number of bytes fed in
0x14	BITS OUT	R	number of compressed bits produced

3.4 Code table

Characters	Code length
space, e	3 bits
t, a, o, i, n, s	4 bits
h, r, d, l	5 bits
c, u, m, w, f, g, y, p, b, new line, full stop, comma	6 bits
v, k	7 bits
digits, x, j, q, z	8 to 11 bits
capital letters, symbols such as ; ( ) { } =	17 to 18 bits

Built from English letter frequencies, the table averages 4.54 bits per character on English text (the script prints this when it runs).

3.5 RTL schematic

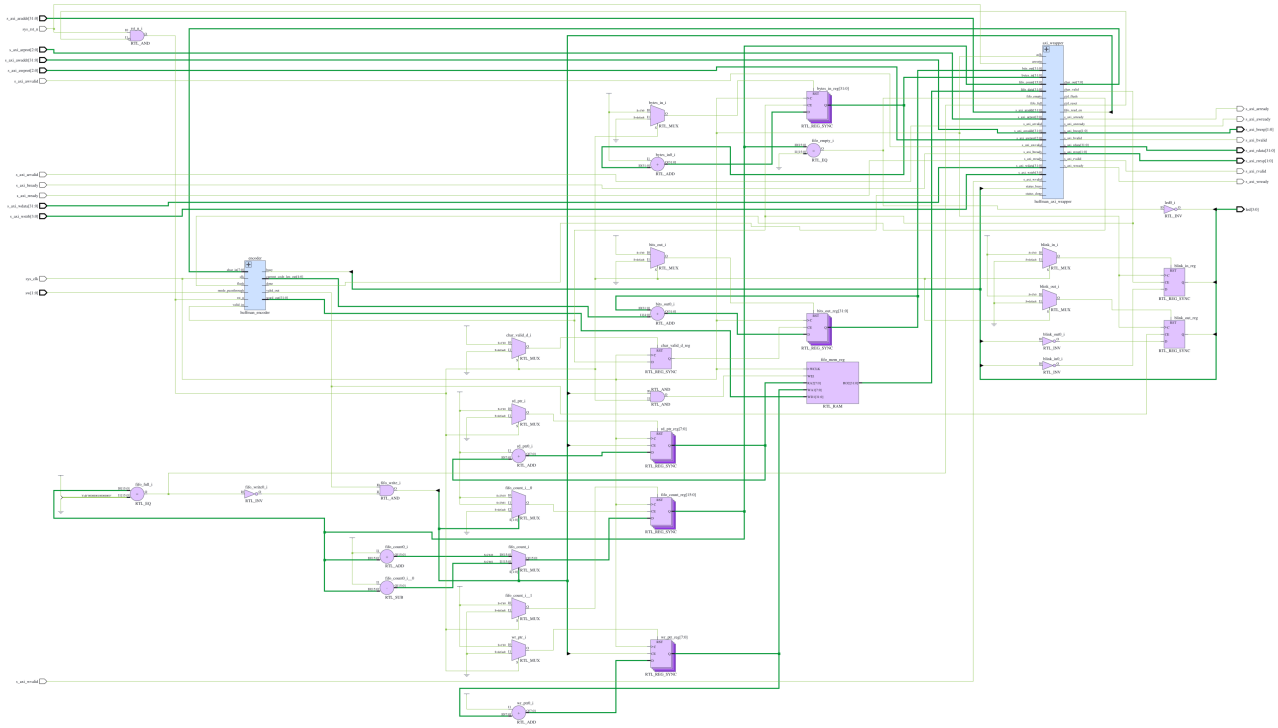


Figure 2 - Elaborated RTL schematic of huffman_top (Vivado)

4. Simulation Results

The design was simulated with Icarus Verilog using the testbench *huffman_tb.sv*. It compresses the 32-byte message "hello world, huffman on a pynq!!", flushes, and checks the byte counter, the bit counter, the number of words and every word against the expected output built from the same code table. All checks passed:

```
bytes in  = 32 (expected 32)
bits out  = 179 (expected 179)
status    = 00060002, 6 words waiting
compressed 32 bytes (179 bits) into 6 words, all correct
PASS: all Huffman accelerator checks passed
```

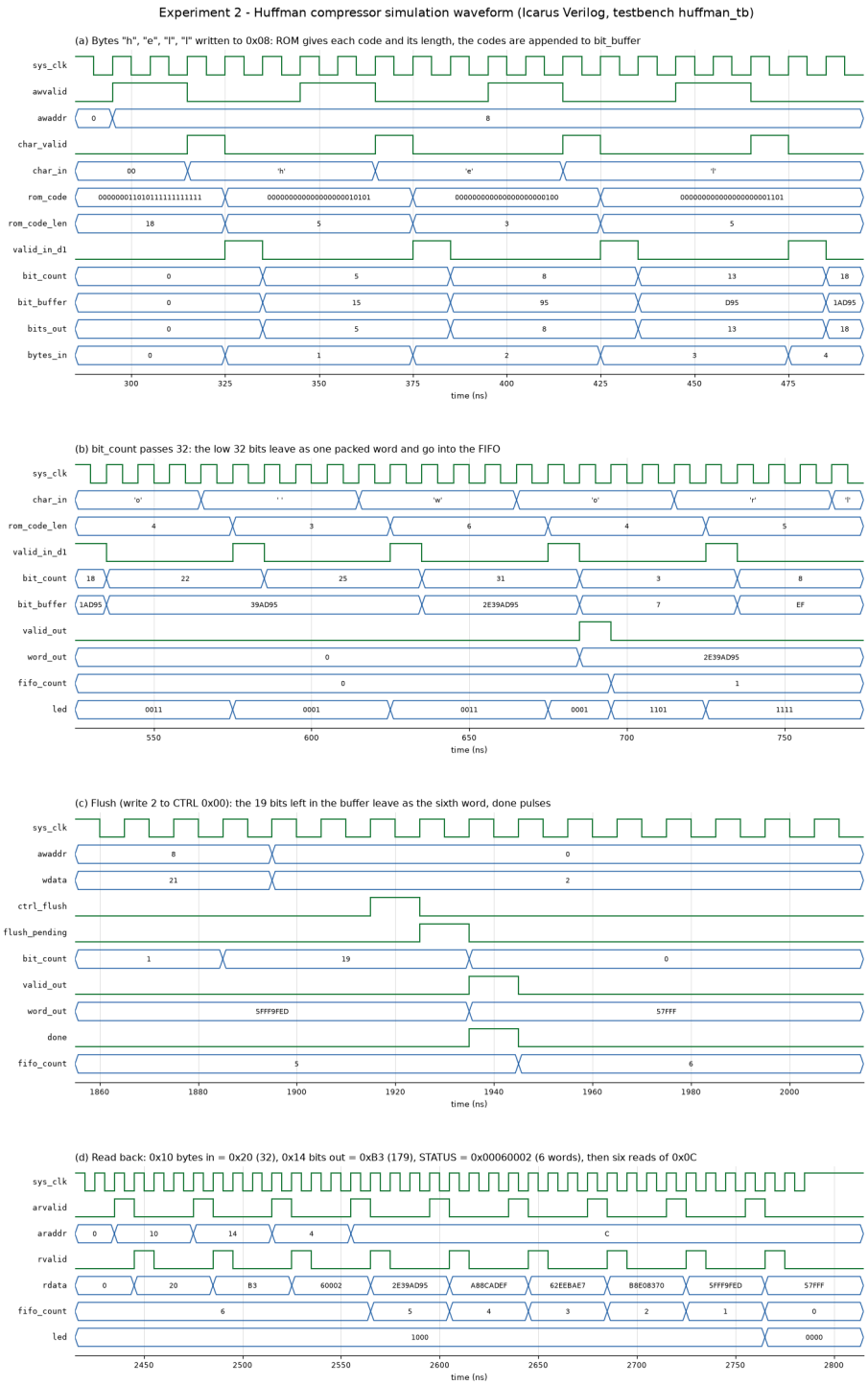


Figure 3 - Simulation waveform (details in Simulation/simulation_report.pdf)

5. Hardware Implementation

5.1 Build

The design was built with Vivado 2026.1 for the xc7z020clg400-1 device using build.tcl. The block design connects the Zynq processing system, a reset block, an AXI interconnect and huffman_top. The constraints file (huffman.xdc) assigns the four LEDs and the two switches.

Resource	Used	Available	Utilisation
Slice LUTs	928	53,200	1.74 %
of which LUT as memory (output FIFO)	236	17,400	1.36 %
Slice registers (flip-flops)	746	106,400	0.70 %
Block RAM tiles	0	140	0 %
Bonded I/O pins	5	125	4.00 %
Global clock buffers	1	32	3.13 %

Timing (100 MHz clock)	Value	Status
Worst setup slack (WNS)	+2.934 ns	**met**
Worst hold slack (WHS)	+0.087 ns	**met**
Failing endpoints	0 of 3185	**met**

5.2 Test on the board

The bitstream was loaded into the PYNQ-Z2 with SW0 down (compression mode). Three texts were sent through the compressor exactly as the notebook huffman_demo.ipynb does it: reset, write every byte to 0x08, read words from 0x0C as they appear, flush and read the counters. For each text the returned words were compared bit for bit with a Python model of the encoder, decoded back to text, and the size was compared with the software compressor zlib at its best setting.

Text	Bytes in	Bits out	Bits / char	FPGA size	zlib size	Result
Notebook text (lower-case English)	600	2764 in 87 words	4.61	57.6 %	21.3 %	**PASS**
English paragraph (normal case)	897	4066 in 128 words	4.53	56.7 %	56.9 %	**PASS**
Verilog source code	584	3449 in 108 words	5.91	73.8 %	42.1 %	**PASS**

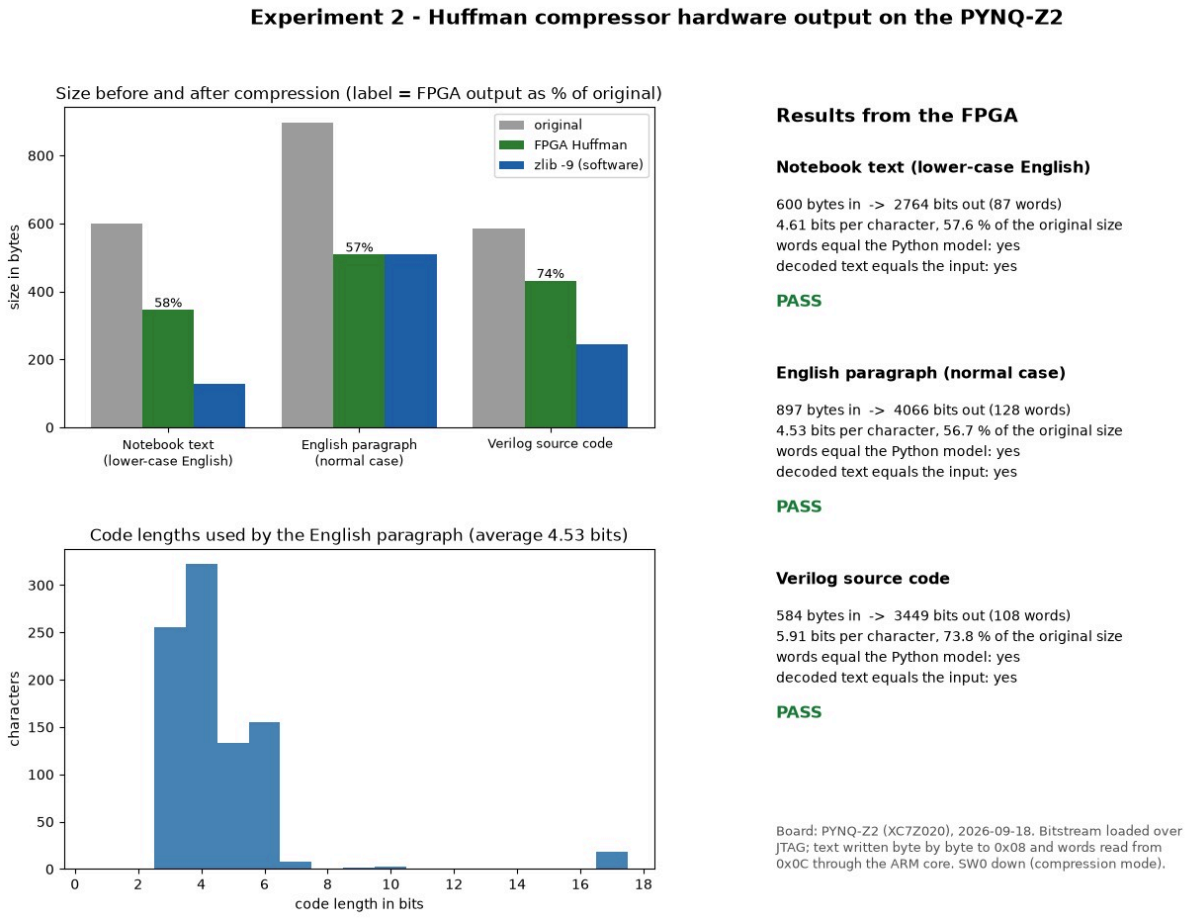


Figure 4 - Compression results measured on the FPGA

Run of the Jupyter notebook on the board

The steps of the notebook huffman_demo.ipynb (Jupyter folder) were also run on the PYNQ-Z2 with PYNQ, using the same bitstream. tools/execute_all_on_pynq.py runs them on the board over SSH and copies the log and the graph back, and tools/embed_nb_outputs.py puts them into the notebook. It sent the same 600-byte text and got back 87 words holding 2764 bits (57.6 % of the original size, 1.74 to 1), exactly the same as the run above, and the decoded text matched the input byte for byte. Through PYNQ the transfer took 0.04 s instead of about 2 s over JTAG. The log and the graph the notebook drew are in Jupyter/output.

```
[*] Flashing FPGA bitstream for Experiment 2 (Huffman)...
[+] FPGA Flashed! DONE LED active.
Overlay loaded, compressor at 0x40000000
Sent 600 bytes (4800 bits) in 0.04 s
Got back 87 words holding 2764 bits
Compression ratio: 1.74 to 1 (57.6 % of original size)
Decoded text starts: the quick brown fox jumps over the lazy dog. huffman coding
PASS: all 600 bytes came back exactly as they went in
```

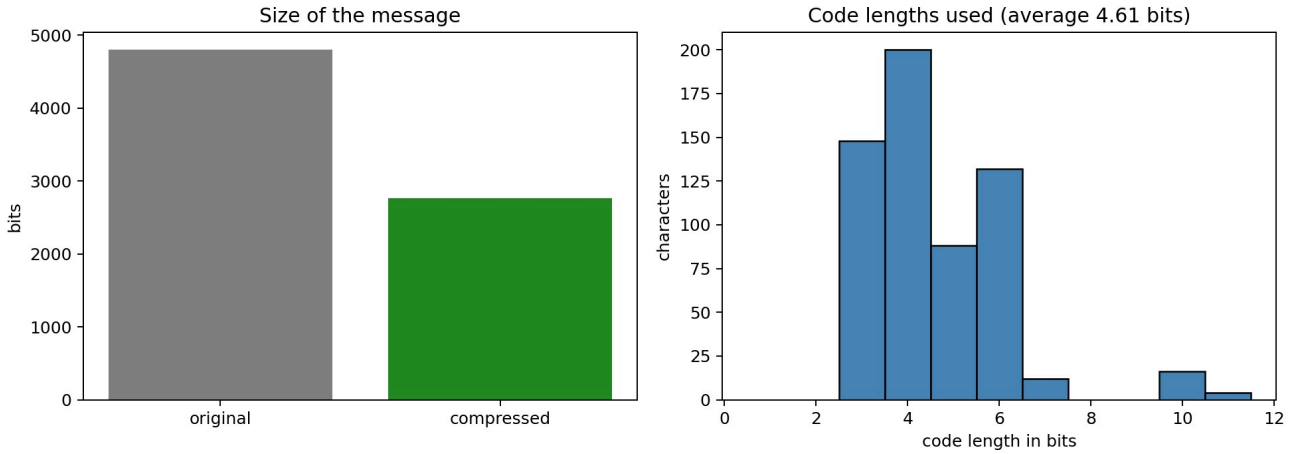


Figure 5 - Graph drawn by huffman_demo.ipynb on the board

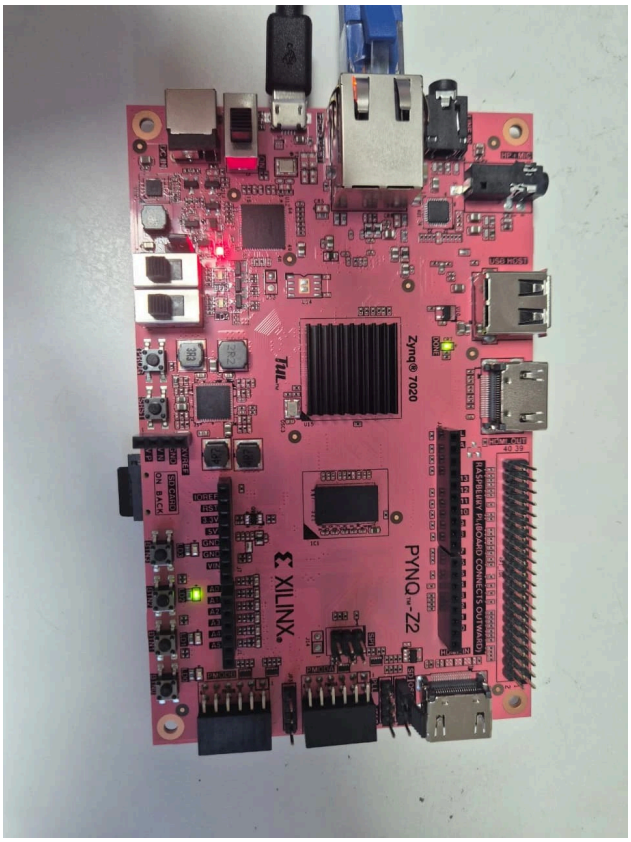


Figure 6 - PYNQ-Z2 board setup: micro-USB (power and JTAG/UART), Ethernet to the PC, PYNQ Linux on the SD card

For all three texts the hardware output was identical to the model and decoded back to exactly the original text. English text comes out at about 4.5 to 4.6 bits per character, 57 % of its original size, which matches the average code length of the table. On the English paragraph the simple FPGA encoder is as good as zlib. zlib is much better on the notebook text because that text repeats the same sentence four times, and zlib also replaces repeated phrases (LZ77), which a fixed Huffman table cannot do. Source code compresses less (74 %) because symbols such as ; ( ) = get 17- or 18-bit codes in a table made for English; this shows that the table should match the kind of data being compressed.

6. Applications

- **IoT and sensor nodes:** compress text logs or messages before sending them over a slow or battery-powered radio link.
- **Data storage and logging:** store more records in the same flash memory.

- **Building block of standard formats:** Huffman coding is the final stage of DEFLATE (ZIP, PNG, gzip) and of JPEG, so the same encoder can be part of a larger hardware compressor.
- **Offloading the processor:** the CPU only moves bytes while the FPGA does the bit-level packing.

7. Conclusion

A Huffman compression accelerator was designed, simulated and implemented on the PYNQ-Z2. It uses under 2 % of the FPGA's LUTs, meets timing at 100 MHz with 2.9 ns to spare, and on the board it compressed English text to about 57 % of its size with the output decoding back perfectly.

Learning outcomes: how Huffman's algorithm builds a prefix-free code; how to generate a hardware lookup table from a script so that hardware, testbench and software always agree; packing variable-length codes into fixed 32-bit words with a shift buffer; writing a FIFO and an AXI4-Lite slave; and checking hardware against a software model.

Future scope: load the code table over AXI so it can be built for each file (adaptive Huffman); add an LZ77 stage in front to remove repeated phrases like zlib does; add a hardware decoder; and use AXI-Stream with DMA so the processor does not have to write every byte itself.

Experiment 3: Pipelined AES-256 Encryption Engine

FPGA Build Challenge 2026 - Intermediate level - AMD PYNQ-Z2 (Zynq-7000 XC7Z020)

1. Objective

AES (Advanced Encryption Standard) is the block cipher used to protect most data today, from Wi-Fi and HTTPS to disk encryption. It works on 128-bit blocks; AES-256 uses a 256-bit key and 14 rounds. Doing AES in software takes the processor many instructions per block. The objective of this experiment is to build AES-256 encryption and decryption as hardware in the FPGA, with one pipeline stage per round so that a new block can enter every clock cycle, and to use it from the ARM processor over AXI4-Lite. The design is checked against the official FIPS-197 test vector and against a software AES library.

2. Block Diagram

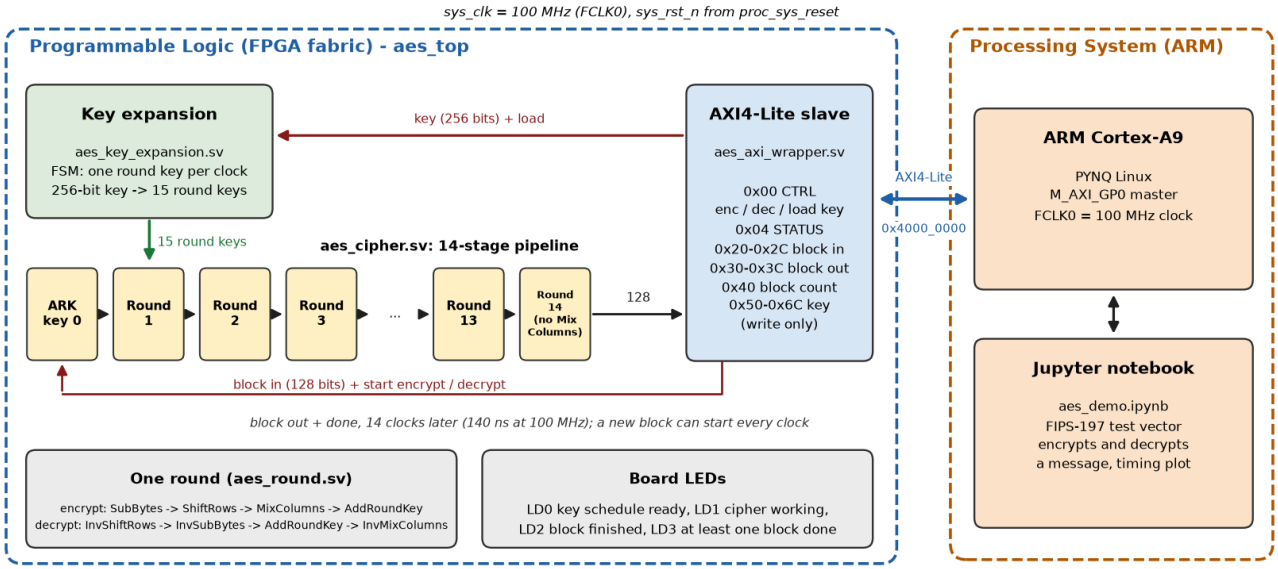


Figure 1 - System block diagram

The processor writes the 256-bit key into write-only registers and starts the key expansion, which computes the 15 round keys (one per clock) and keeps them in registers. It then writes a 128-bit block and starts an encryption or a decryption. The block is XORed with the first round key and passes through 14 round stages; each stage is one clock, so the result appears 14 clocks (140 ns) later. The AXI wrapper catches the result when the last stage signals valid, and the processor reads it back.

3. RTL Design

3.1 Module hierarchy

```
system_wrapper                (Vivado block design)
 |-- processing_system7_0     ARM Cortex-A9, gives FCLK0 = 100 MHz and M_AXI_GP0
 |-- rst_ps7_0_100M           reset synchroniser
 |-- ps7_0_axi_periph         AXI interconnect / protocol converter
 +-- aes_top_0                aes_top.sv  (block counter, LEDs)
      |-- u_axi_wrapper       aes_axi_wrapper.sv
      |-- u_key_expansion     aes_key_expansion.sv  (4 x aes_sbox)
      +-- u_cipher            aes_cipher.sv
           |-- gen_round[1..13].u_round   aes_round.sv (with MixColumns)
           +-- u_final_round              aes_round.sv (no MixColumns)
                |-- aes_sub_bytes  -> 16 x aes_sbox
                |-- aes_shift_rows
                |-- aes_mix_columns
                +-- (AddRoundKey = XOR with the round key)
```

3.2 Modules

Module (file)	What it does
aes_sbox (aes_sbox.sv)	The AES S-box and inverse S-box as 256-entry lookup tables: one byte in, one byte out.
aes_sub_bytes (aes_sub_bytes.sv)	Applies the S-box (or inverse S-box) to all 16 bytes of the state.
aes_shift_rows (aes_shift_rows.sv)	Rotates row r of the 4x4 byte state left by r bytes (right for decryption). Only wiring, no logic.
aes_mix_columns (aes_mix_columns.sv)	Multiplies each column by a fixed matrix in GF(2^8) using "xtime" (shift and conditional XOR with 0x1B); the inverse matrix is used for decryption.
aes_add_round_key	XOR of the state with the 128-bit round key.
aes_round (aes_round.sv)	One round, with both paths built in parallel: encrypt = SubBytes, ShiftRows, MixColumns, AddRoundKey; decrypt = InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns. A multiplexer picks one and a 128-bit register stores it: this register is the pipeline stage. The final round has no MixColumns.
aes_cipher (aes_cipher.sv)	Initial AddRoundKey, 13 normal rounds and the final round chained as a 14-stage pipeline. The encrypt/decrypt flag travels down the pipeline with the data, and decryption uses the round keys in reverse order.
aes_key_expansion (aes_key_expansion.sv)	State machine (IDLE, EXPAND, READY). Round keys 0 and 1 are the two halves of the key; keys 2 to 14 are computed one per clock with RotWord, SubWord (4 S-boxes) and the Rcon constant, as in FIPS-197.
aes_axi_wrapper (aes_axi_wrapper.sv)	AXI4-Lite slave: control pulses, status, the 4 input words, 4 result words (captured when the cipher signals done), the block counter and 8 key words that read back as zero.
aes_top (aes_top.sv)	Connects the blocks, only starts the cipher when a key schedule is ready, counts finished blocks and stretches the busy/done pulses to about 84 ms so they are visible on LD1 and LD2.

3.3 Register map (base address 0x4000_0000)

Offset	Name	Access	Meaning
0x00	CTRL	W	bit 0 start encrypt, bit 1 start decrypt, bit 2 load the key
0x04	STATUS	R	bit 0 busy, bit 1 done, bit 2 key schedule ready
0x20-0x2C	BLOCK IN	R/W	128-bit input block, most significant word first
0x30-0x3C	BLOCK OUT	R	128-bit result
0x40	BLOCKS	R	number of blocks processed
0x50-0x6C	KEY	W	256-bit key, most significant word first; reads return 0

3.4 RTL schematics

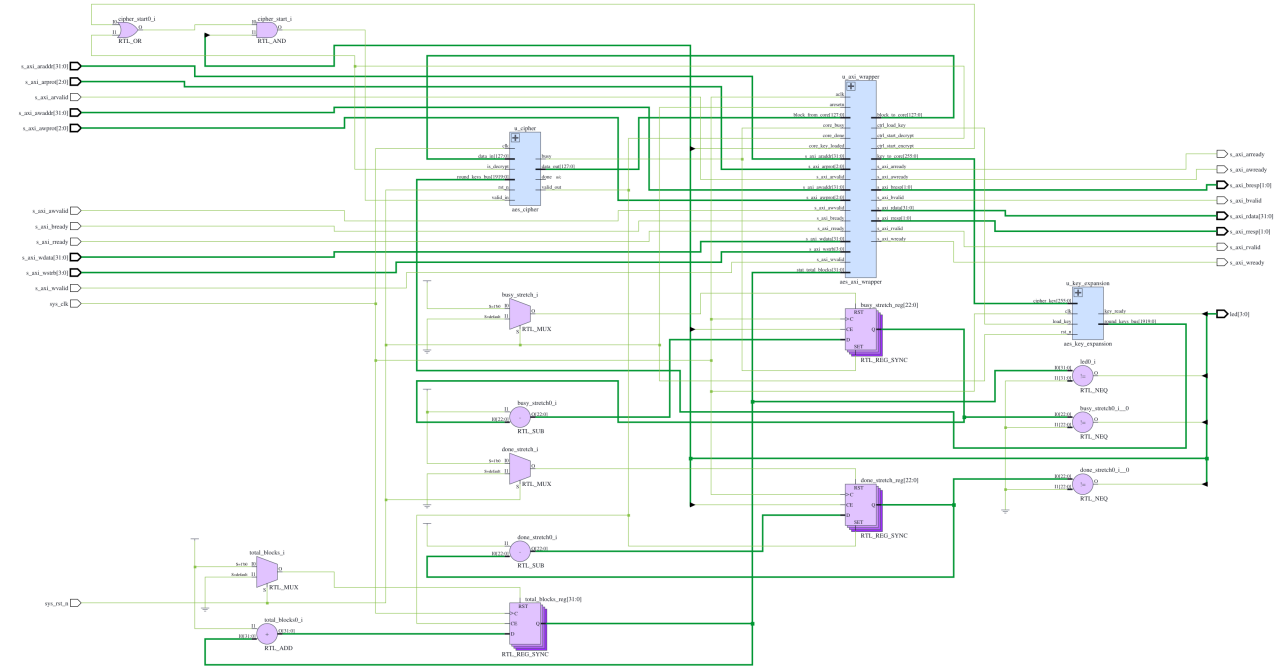


Figure 2 - Elaborated RTL schematic of aes_top (Vivado)

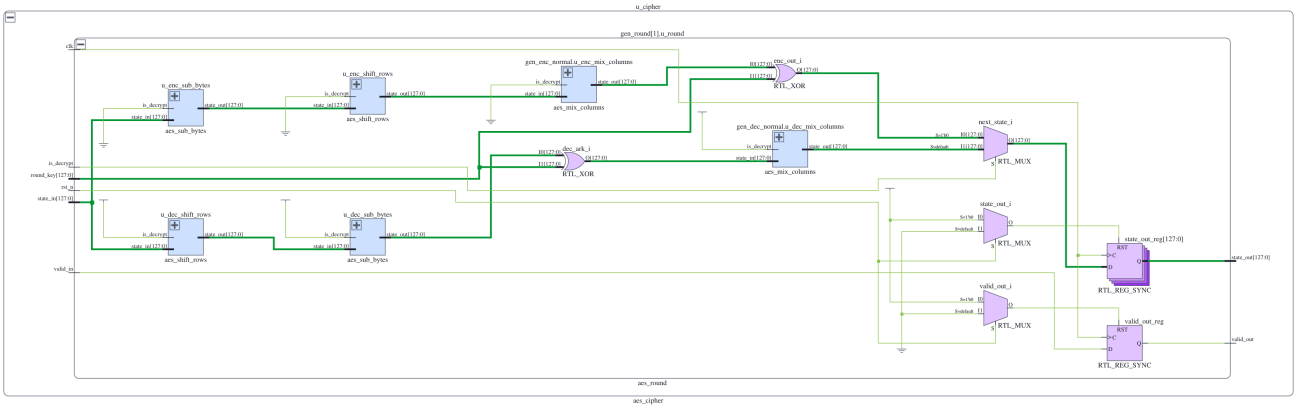


Figure 3 - One round (gen_round[1]): encrypt path (top) and decrypt path (bottom) feed the pipeline register

4. Simulation Results

The design was simulated with Icarus Verilog using the testbench *aes_tb.sv* with the FIPS-197 appendix C.3 AES-256 example. All checks passed:

```
key schedule ready (status = 00000004)
plaintext  00112233445566778899aabbccddeeff
ciphertext 8ea2b7ca516745bfeafc49904b496089
[PASS] encryption matches the FIPS-197 test vector
decrypted  00112233445566778899aabbccddeeff
[PASS] decryption recovered the original block
[PASS] the key registers read back as zero
blocks processed: 2
```

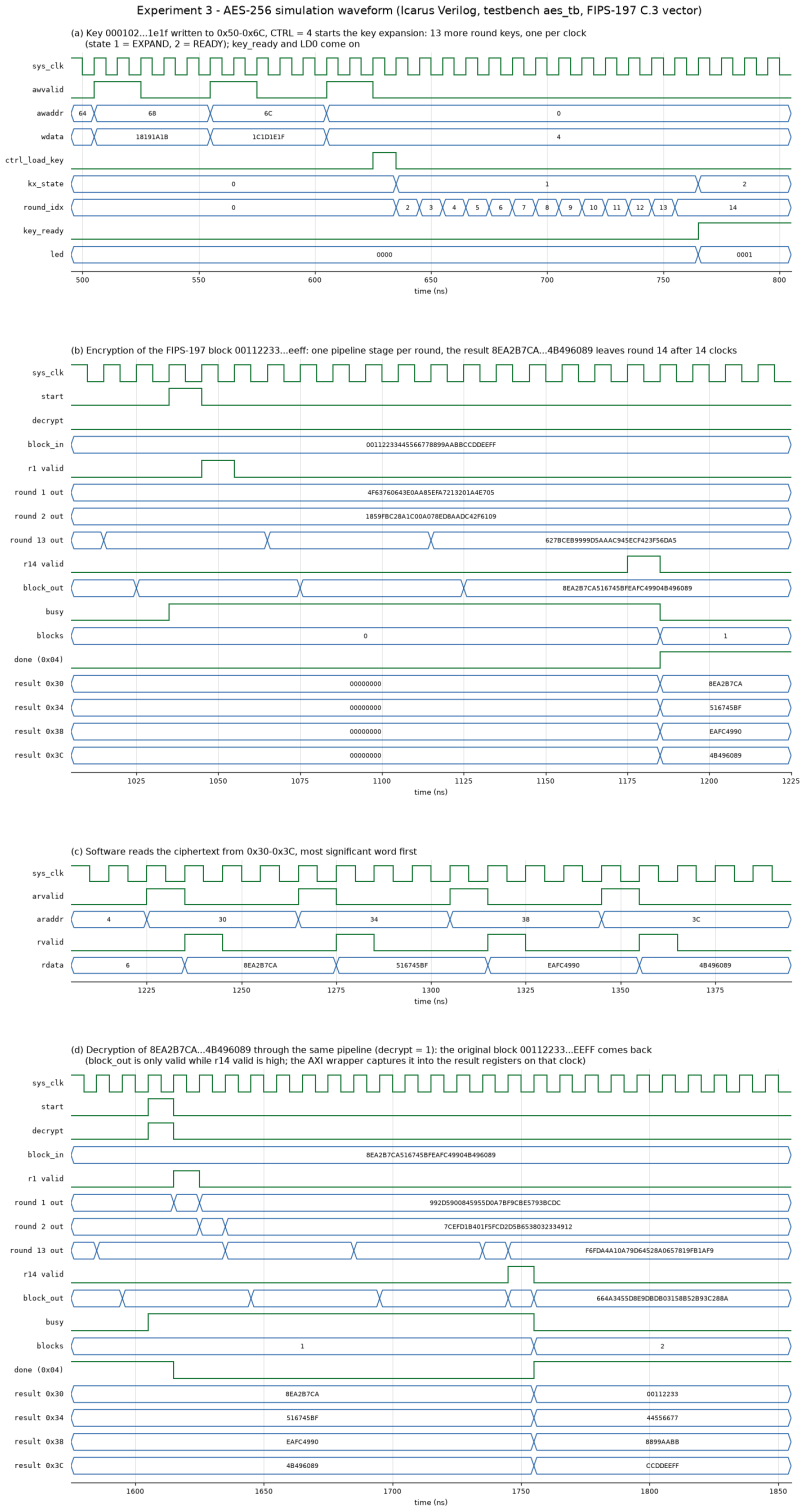


Figure 4 - Simulation waveform (details in Simulation/simulation_report.pdf)

5. Hardware Implementation

5.1 Build

The design was built with Vivado 2026.1 for the xc7z020clg400-1 device using build.tcl. The constraints file (aes.xdc) assigns the four LEDs. The 14 unrolled rounds each contain 16 S-boxes for encryption and 16 inverse S-boxes for decryption plus the 4 S-boxes of the key expansion - about 450 lookup tables of 256 bytes - so the design uses about half of the FPGA's LUTs, most of them (with the F7/F8 multiplexers) for the S-boxes.

Resource	Used	Available	Utilisation
Slice LUTs	26,763	53,200	50.31 %
Slice registers (flip-flops)	4,922	106,400	4.63 %
F7 / F8 multiplexers	5,233 / 804	26,600 / 13,300	19.67 % / 6.05 %
Block RAM tiles	0	140	0 %
Bonded I/O pins	4	125	3.20 %
Global clock buffers	1	32	3.13 %

Timing (100 MHz clock)	Value	Status
Worst setup slack (WNS)	+0.273 ns	**met**
Worst hold slack (WHS)	+0.010 ns	**met**
Failing endpoints	0 of 12,533	**met**

Each pipeline stage does a full round (S-box, MixColumns and XOR) in one 10 ns clock, which is why the setup slack is small but positive.

5.2 Test on the board

The bitstream was loaded into the PYNQ-Z2 and the engine was driven through its AXI registers exactly as the notebook aes_demo.ipynb does it. The results were compared with the Python "cryptography" library (AES-256, ECB mode). 120 blocks were processed in total.

Check	Detail	Result
FIPS-197 C.3 encryption	got 8ea2b7ca516745bfeafc49904b496089	**PASS**
FIPS-197 C.3 decryption	got 00112233445566778899aabbccddeeff	**PASS**
Key registers read back as zero	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000	**PASS**
Message ciphertext equals software AES	3 blocks	**PASS**
Message decrypts back to the text	'FPGA hardware encryption on the PYNQ-Z2 board!!!'	**PASS**
Random blocks encrypted correctly	40 of 40	**PASS**
Random blocks decrypted correctly	40 of 40	**PASS**
Avalanche blocks equal software AES	32 blocks	**PASS**
Block counter	120 blocks counted, 120 sent	**PASS**

Avalanche effect: flipping one plaintext bit changed on average 65.7 of 128 ciphertext bits (min 52, max 78, 32 flips). A good cipher changes about half of the output bits when one input bit changes, so that similar messages give unrelated ciphertexts.

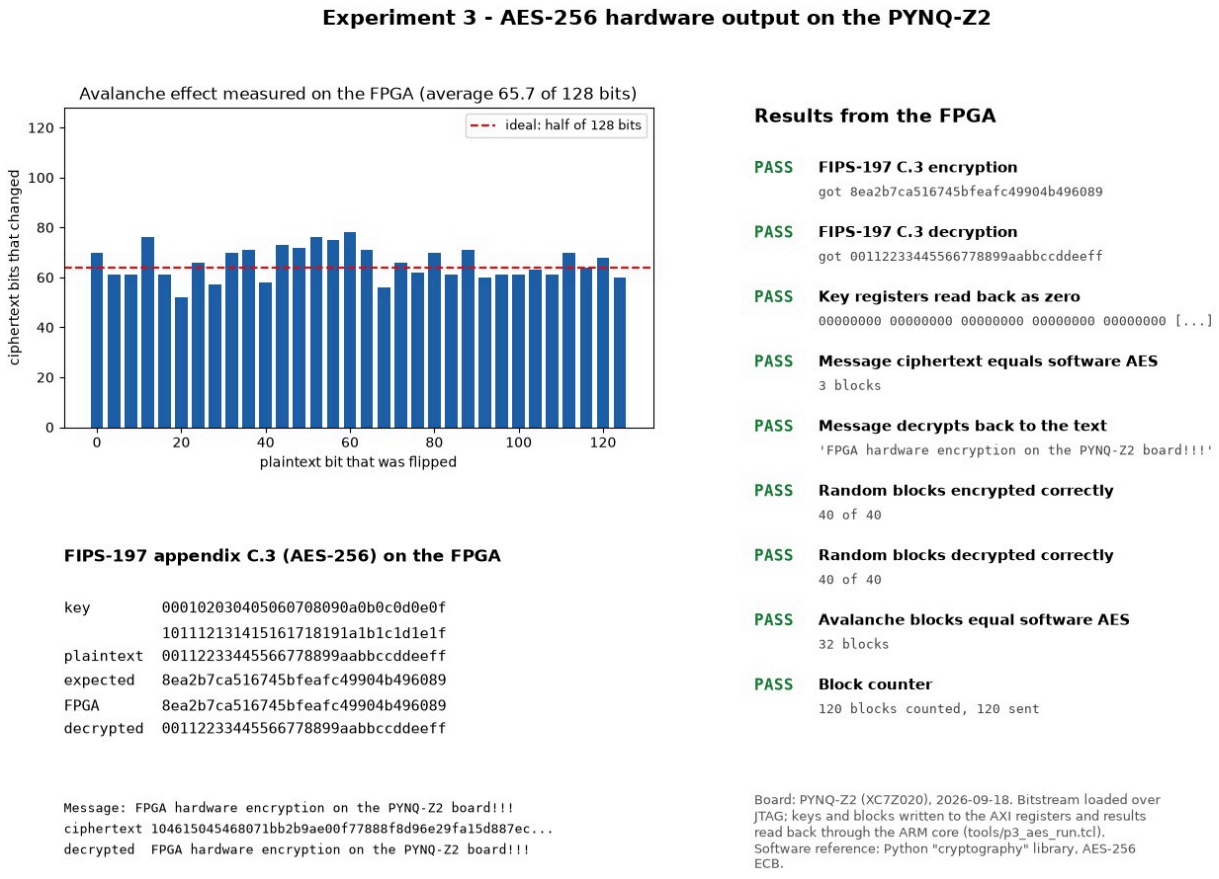


Figure 5 - AES-256 results measured on the FPGA

Run of the Jupyter notebook on the board

The steps of the notebook aes_demo.ipynb (Jupyter folder) were also run on the PYNQ-Z2 with PYNQ, using the same bitstream. tools/execute_all_on_pynq.py runs them on the board over SSH and copies the log and the graph back, and tools/embed_nb_outputs.py puts them into the notebook. It repeated the FIPS-197 test (ciphertext 8ea2b7ca516745bfeafc49904b496089, decryption returns the plaintext) and encrypted and decrypted the 48-byte message (3 blocks), which came back unchanged. Through PYNQ one block took 0.27 ms including the Python register accesses; the cipher itself needs only 140 ns. The log and the graph the notebook drew are in Jupyter/output.

```
[*] Flashing FPGA bitstream for Experiment 3 (AES)...
[+] FPGA Flashed! DONE LED active.
Overlay loaded, AES engine at 0x40000000
plaintext   00112233445566778899aabbccddeeff
ciphertext  8ea2b7ca516745bfeafc49904b496089
expected    8ea2b7ca516745bfeafc49904b496089
decrypted   00112233445566778899aabbccddeeff
encryption matches FIPS-197: PASS
decryption returns the input: PASS
message   : FPGA hardware encryption on the PYNQ-Z2 board!!!
encrypted : 104615045468071bb2b9ae00f77888f8d96e29fa15d887ecfe101d854140b432be385b36418697c76e21e0d940642217
decrypted : FPGA hardware encryption on the PYNQ-Z2 board!!!
round trip: PASS
3 blocks encrypted in 0.8 ms (0.27 ms per block)
blocks processed since power up: 8
```

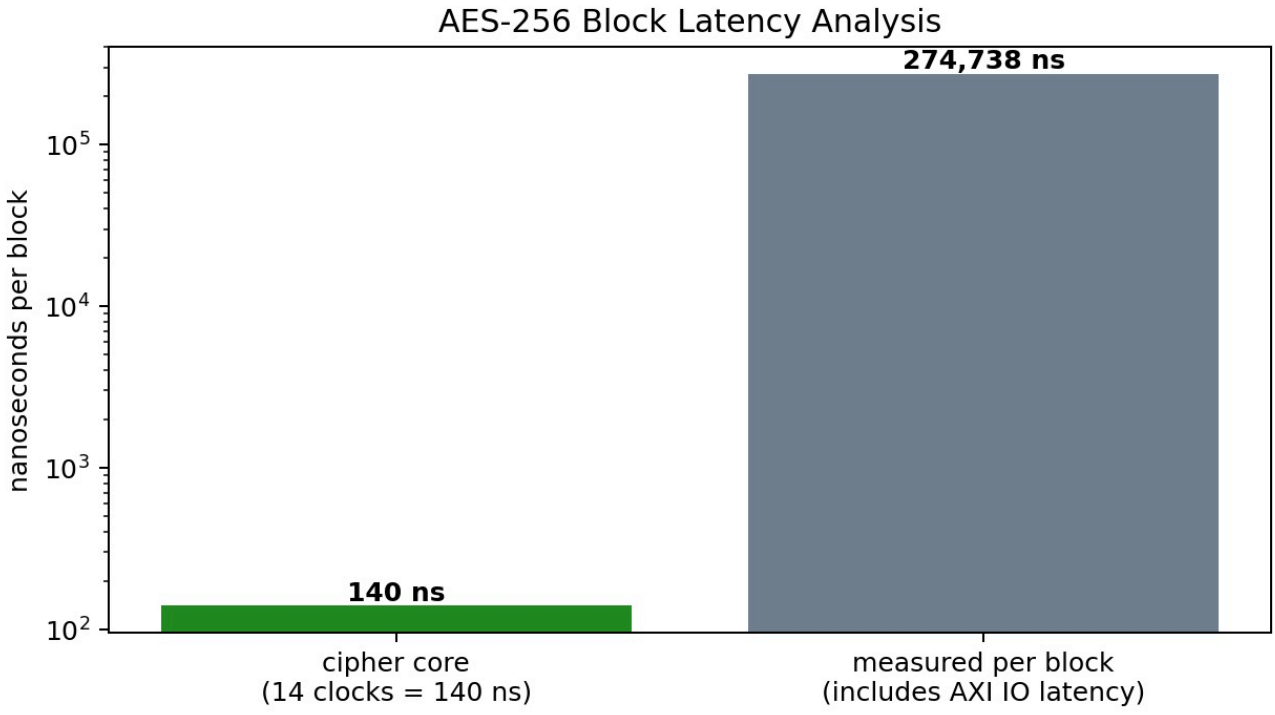


Figure 6 - Graph drawn by aes_demo.ipynb on the board

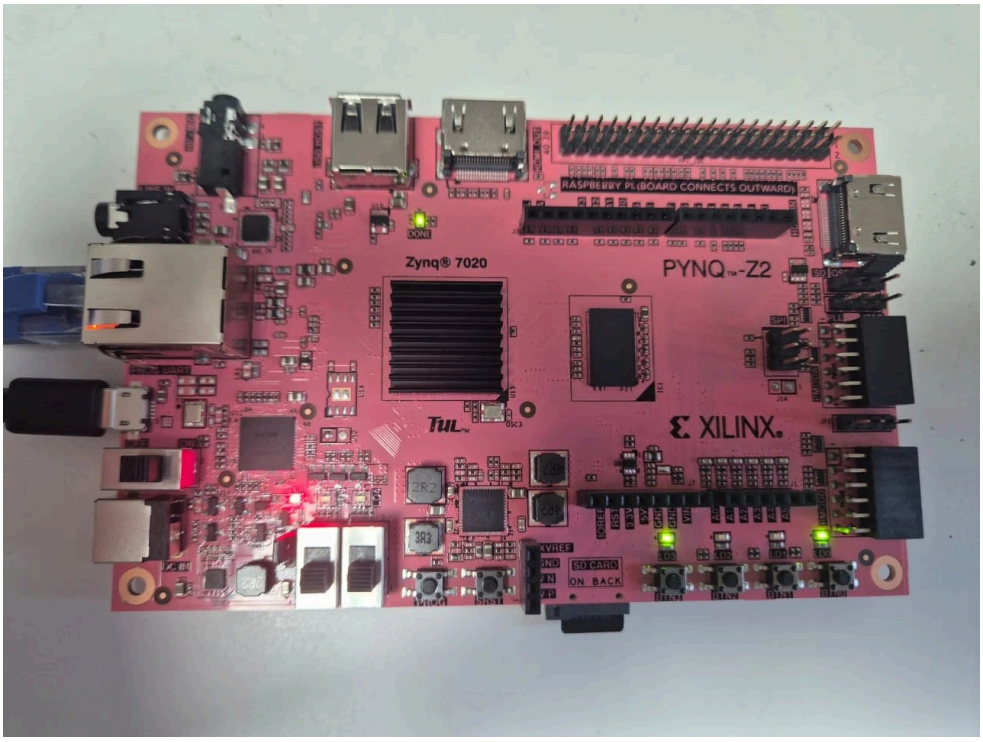


Figure 7 - PYNQ-Z2 board setup: micro-USB (power and JTAG/UART), Ethernet to the PC, PYNQ Linux on the SD card

The cipher itself needs 14 clocks (140 ns) per block, which is about 114 MB/s for one block at a time and up to 1.6 GB/s if a new block were started every clock. Through AXI4-Lite the processor has to write four words, start the engine, poll the status and read four words for every block, so in practice the speed is set by these register accesses, not by the cipher.

6. Applications

- **Secure communication:** encrypting network packets (VPN, TLS, Wi-Fi WPA2/WPA3 use AES) at line rate without loading the processor.
- **Storage encryption:** encrypting disks, SD cards and flash in embedded systems.
- **IoT and sensor networks:** protecting sensor readings before they are sent over radio.
- **Secure boot and firmware protection:** decrypting firmware images inside the device.

7. Conclusion

A pipelined AES-256 engine was designed, simulated and implemented on the PYNQ-Z2. It meets timing at 100 MHz, produces the FIPS-197 reference result, matched a software AES library on all 120 test blocks on the board, and keeps the key write-only.

Learning outcomes: the structure of AES (SubBytes, ShiftRows, MixColumns, AddRoundKey and the key schedule); arithmetic in GF(2^8); how pipelining trades area for throughput (one round per stage, a new block every clock); reusing one datapath for encryption and decryption; and verifying hardware against a standard test vector and a software reference.

Future scope: add AXI-Stream and DMA so that blocks can really enter every clock; add block cipher modes such as CTR or GCM in hardware (ECB alone is not secure for long messages); save area by sharing S-boxes or by building only the encryption path; and add protection against side-channel attacks.

Experiment 4: NeuroCore-32 RISC-V Processor

FPGA Build Challenge 2026 - Intermediate level - AMD PYNQ-Z2 (Zynq-7000 XC7Z020)

1. Objective

RISC-V is an open instruction set that anyone may implement. The objective of this experiment is to design a 32-bit RISC-V processor for the RV32I base instruction set in the FPGA fabric, using the classic five-stage pipeline (fetch, decode, execute, memory, write back) so that up to five instructions are worked on at once. The pipeline is kept correct and fast by a forwarding unit, a hazard unit and a branch predictor, and hardware counters measure how well it performs. The ARM processor loads programs into the core over AXI4-Lite, starts it and reads the results back. Small programs - Fibonacci numbers, a bubble sort and an IoT sensor alarm - show it working on the board.

2. Block Diagram

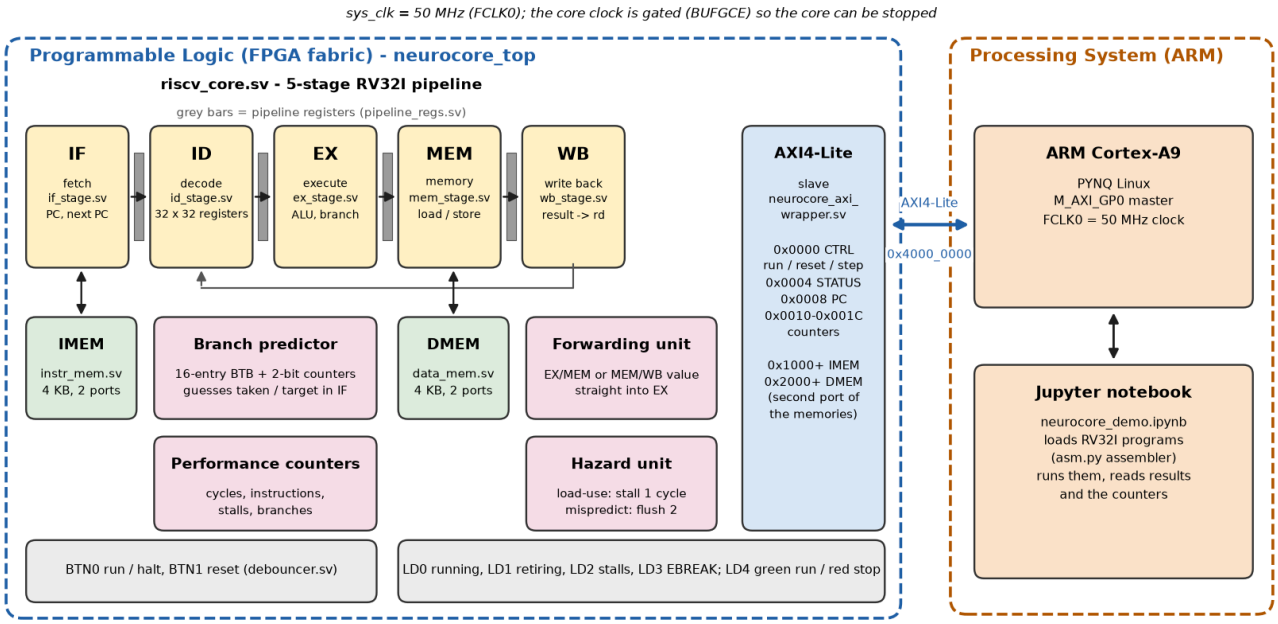


Figure 1 - System block diagram

The ARM writes a program into the 4 KB instruction memory and any input data into the 4 KB data memory through the second port of each memory, then sets the run bit. The core fetches, decodes and executes the program on its own until it reaches EBREAK, which stops the core clock and sets the halt flag. The ARM then reads the results from data memory and the performance counters (cycles, instructions retired, stalls, branches) from the status registers.

3. RTL Design

3.1 Module hierarchy

```
system_wrapper                  (Vivado block design, FCLK0 = 50 MHz)
 +-- neurocore_top_0            neurocore_top.sv  (buttons, LEDs, clock gate)
      |-- u_deb_btn0, u_deb_btn1  debouncer.sv
      |-- u_axi_wrapper          neurocore_axi_wrapper.sv
      +-- u_riscv_core           riscv_core.sv
           |-- if_stage          if_stage.sv + instr_mem.sv
           |-- id_stage          id_stage.sv (decoder, 32 x 32-bit register file)
           |-- ex_stage          ex_stage.sv (ALU, branch compare and target)
           |-- mem_stage         mem_stage.sv + data_mem.sv
           |-- wb_stage          wb_stage.sv
           |-- if_id / id_ex / ex_mem / mem_wb registers   pipeline_regs.sv
           |-- forwarding_unit   forwarding_unit.sv
           |-- hazard_unit       hazard_unit.sv
           |-- branch_predictor  branch_predictor.sv
           +-- perf_counters     perf_counters.sv
defines.sv                      opcodes, ALU operations and the pipeline register types
```

3.2 Modules

Module (file)	What it does
if_stage, instr_mem	Holds the PC and fetches the instruction. The next PC is PC + 4, the predicted target from the branch predictor, or the correct address after a misprediction. The instruction memory is 1024 x 32 bits with a second port for the ARM.
id_stage	Decodes the instruction (R, I, S, B, U and J formats of RV32I), builds the immediate, reads two registers from the 32 x 32-bit register file (x0 is always 0) and produces the control signals.
ex_stage	ALU (add, sub, and, or, xor, shifts, set-less-than), compares for branches and computes the branch or jump target. Its inputs can come from the forwarding paths.
mem_stage, data_mem	Loads and stores words, half-words and bytes in the 1024 x 32-bit data memory, which has a second port for the ARM.
wb_stage	Chooses the ALU result, the loaded value or PC + 4 (for jumps) and writes it to register rd.
pipeline_regs	The IF/ID, ID/EX, EX/MEM and MEM/WB registers, with stall (hold) and flush (clear) inputs.
forwarding_unit	If an instruction in EX needs a register that an older instruction in MEM or WB is about to write, the value is taken from that stage instead of the register file.
hazard_unit	Load-use hazard (a load followed by an instruction that needs the loaded value): stall one cycle. Branch mispredicted in EX: flush the two instructions fetched after it.
branch_predictor	16-entry branch target buffer with a 2-bit saturating counter per entry, looked up with the fetch PC. A hit with the counter in a "taken" state sends fetch straight to the stored target.
perf_counters	Counts clock cycles, retired instructions, stall cycles and branches.
neurocore_axi_wrapper	AXI4-Lite slave: control (run, reset, single step), status, PC, current instruction, the four counters and the two memory windows. A BUFGCE gates the core clock, so the core stops at EBREAK.
neurocore_top	Joins the core and the wrapper, debounces BTN0 (run / halt) and BTN1 (reset) and drives the LEDs.

3.3 Register map (base address 0x4000_0000)

Offset	Name	Access	Meaning
0x0000	CTRL	W	bit 0 run, bit 1 reset, bit 2 single step
0x0004	STATUS	R	bit 0 running, bit 1 stopped at EBREAK
0x0008 / 0x000C	PC / INSTR	R	program counter and current instruction
0x0010 - 0x001C	COUNTERS	R	cycles, instructions retired, stalls, branches
0x1000 +	IMEM	W	instruction memory (written while the core is stopped)
0x2000 +	DMEM	R/W	data memory

3.4 Test programs

The programs are written with a small Python assembler (Testbench/programs/asm.py) that encodes RV32I instructions and resolves labels, and are saved as .hex files.

Program	Size	What it does
fibonacci.hex	17 instructions	Computes the first 20 Fibonacci numbers and stores them at 0x100.
bubble_sort.hex	29 instructions	Writes 15 ... 0 into memory at 0x000 and sorts them with nested loops (loads, stores and many branches).
sensor_threshold.hex	23 instructions	Reads 16 temperatures from 0x000, writes an alarm flag (1 if the reading is 45 C or more) for each to 0x100 and the number of alarms to 0x180 - a simple IoT edge task.

3.5 RTL schematics

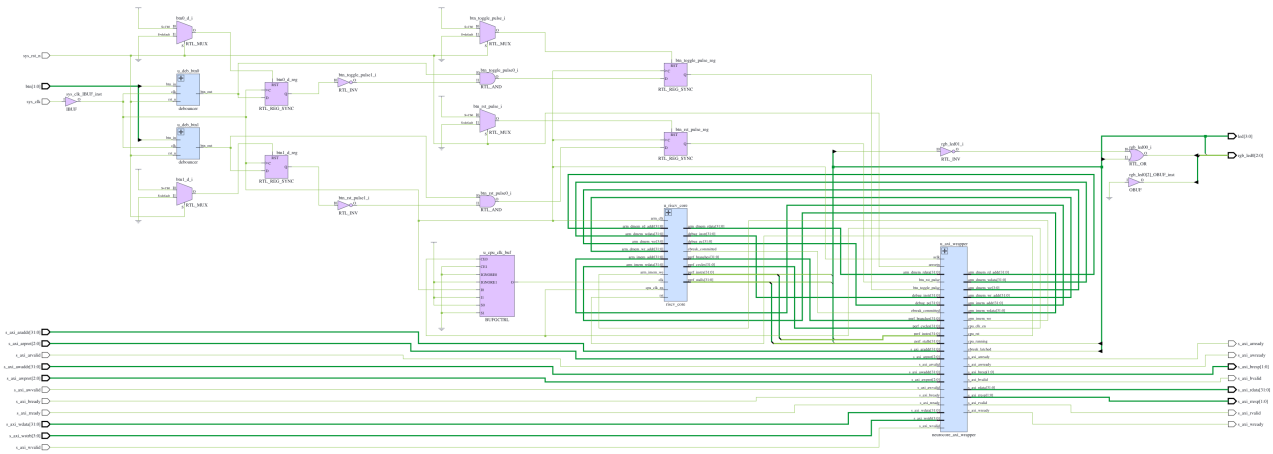


Figure 2 - Elaborated RTL schematic of neurocore_top (Vivado)

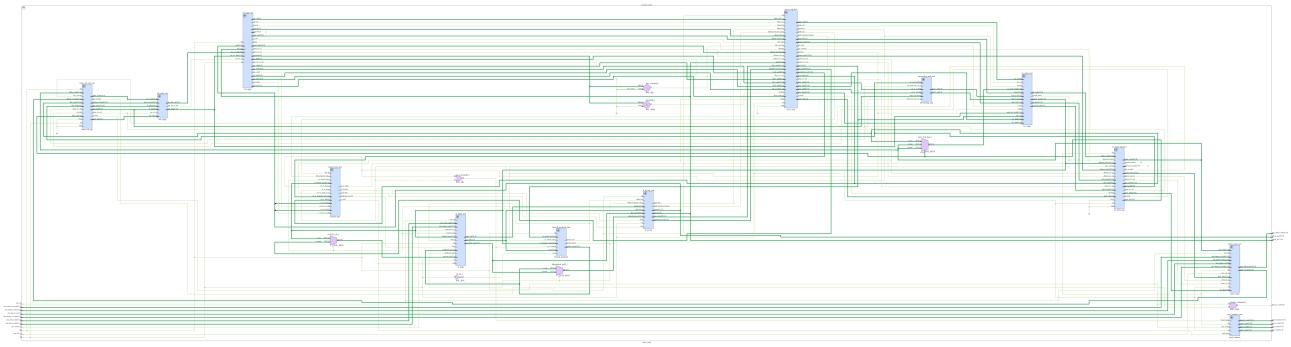


Figure 3 - Inside riscv_core: the five stages, pipeline registers, forwarding, hazard and branch-prediction units (full size in Images/rtl_schematic_core.png)

4. Simulation Results

The design was simulated with Icarus Verilog using the testbench *neurocore_pynq_tb.sv*, which executes four verification phases: Fibonacci sequence generation, bubble sort, IoT sensor threshold detection, and an EBREAK regression test. All checks passed:

```
loaded fibonacci.hex (17 instructions)
program stopped at EBREAK (status 00000002)
[PASS] the first 20 Fibonacci numbers are correct
  cycles 179, instructions 171, stalls 0, branches 37
  IPC: 0.955
loaded bubble_sort.hex (29 instructions)
program stopped at EBREAK (status 00000002)
[PASS] the array was sorted into 0 to 15
loaded sensor_threshold.hex (23 instructions)
program stopped at EBREAK (status 00000002)
[PASS] sensor alarm flags are correct
loaded fibonacci.hex (17 instructions)
program stopped at EBREAK (status 00000002)
[PASS] EBREAK regression: F15 = 610 (post-sensor store was correctly suppressed)
PASS: NeuroCore-32 ran all programs correctly
```

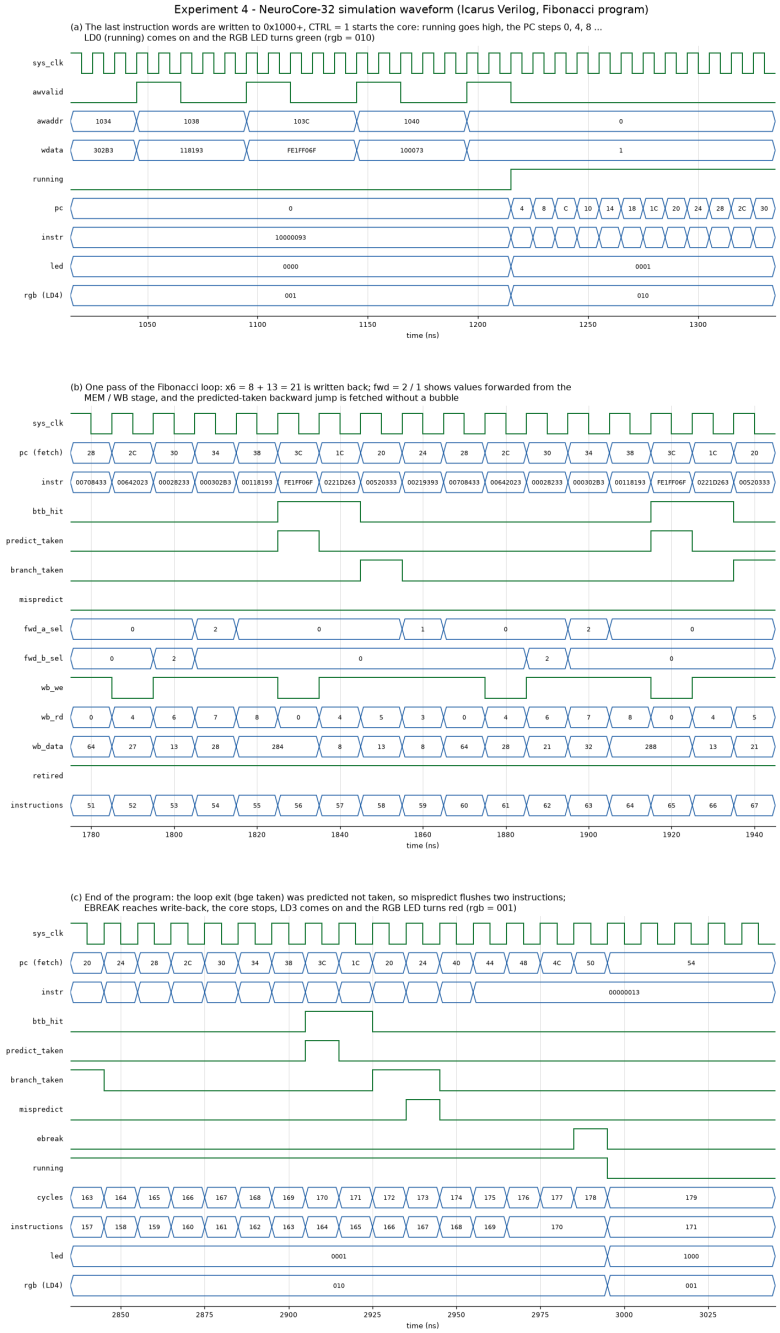


Figure 4 - Simulation waveform (details in Simulation/simulation_report.pdf)

5. Hardware Implementation

5.1 Build

The design was built with Vivado 2026.1 for the xc7z020clg400-1 device using build.tcl, with the fabric clock set to 50 MHz. The constraints file (pynq_z2.xdc) assigns the four LEDs, the RGB LED LD4 and the two buttons. The two 4 KB memories are built from LUTs (distributed RAM), which is most of the "LUT as memory" figure.

Resource	Used	Available	Utilisation
Slice LUTs	4,801	53,200	9.02 %
of which LUT as memory (IMEM, DMEM)	2,344	17,400	13.47 %
Slice registers (flip-flops)	2,347	106,400	2.21 %
Block RAM tiles	0	140	0.00 %
Bonded I/O pins	9	125	7.20 %
Global clock buffers (incl. the core clock gate)	2	32	6.25 %

Timing (50 MHz clock)	Value	Status
Worst setup slack (WNS)	+4.104 ns	**met**
Worst hold slack (WHS)	+0.022 ns	**met**
Timing endpoints	23,851	

5.2 Test on the board

The bitstream was loaded into the PYNQ-Z2 and the three programs were run exactly as the notebook neurocore_demo.ipynb does it: reset the core, write the program (and, for the sensor program, 16 temperature readings) through the AXI memory windows, start it, wait for EBREAK and read the results and the counters.

Check	Detail	Result
Fibonacci: first 20 numbers	F19 = 4181	**PASS**
Fibonacci: counters equal simulation	179 cycles, 171 instructions (sim: 179, 171)	**PASS**
Bubble sort: array is 0 to 15	0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15	**PASS**
Sensor: alarm flags	0011010101100101	**PASS**
Sensor: anomaly count	8 readings >= 45 C	**PASS**
All programs stopped at EBREAK	STATUS = 0x00000002	**PASS**

Program	Cycles	Instructions	Stalls	Branches	IPC	Time at 50 MHz
Fibonacci	179	171	0	37	0.955	3.58 us
Bubble sort	1575	1411	120	439	0.896	31.50 us
Sensor threshold	224	186	0	57	0.830	4.48 us

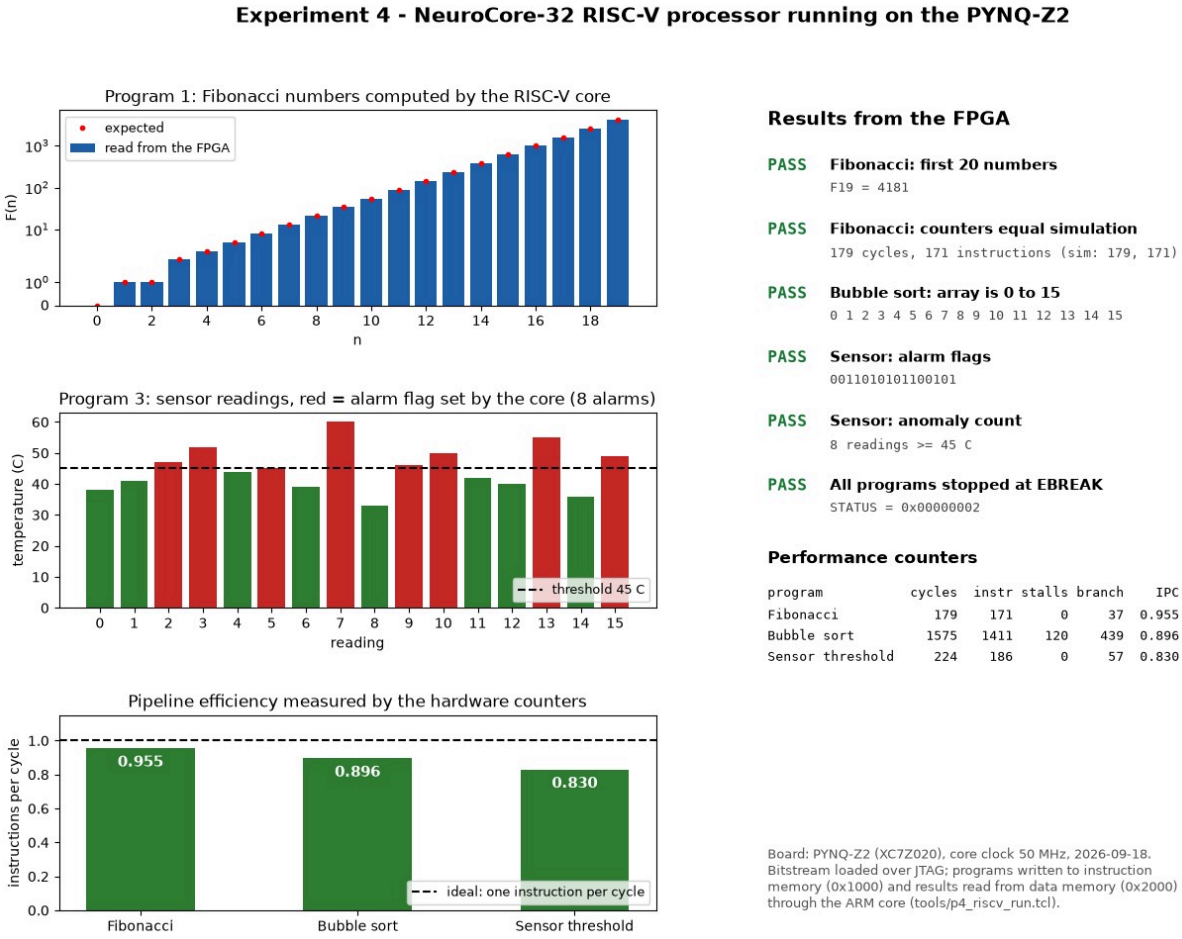


Figure 5 - Programs run by the RISC-V core on the FPGA

Run of the Jupyter notebook on the board

The steps of the notebook neurocore_demo.ipynb (Jupyter folder) were also run on the PYNQ-Z2 with PYNQ, using the same bitstream. tools/execute_all_on_pynq.py runs them on the board over SSH and copies the log and the graph back, and tools/embed_nb_outputs.py puts them into the notebook. It ran the Fibonacci and bubble sort programs. Both results are correct, and the counters are exactly the same as in the run above and in simulation: 171 instructions in 179 cycles (IPC 0.955) for Fibonacci, and 1411 instructions in 1575 cycles with 120 stalls and 439 branches for the sort. The log and the graph the notebook drew are in Jupyter/output.

```
[*] Flashing FPGA bitstream for Experiment 4 (Neurocore RISC-V)...
[+] FPGA Flashed! DONE LED active.
Overlay loaded, processor at 0x40000000
Fibonacci numbers from the FPGA:
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181]
correct: PASS
171 instructions in 179 cycles = 0.955 IPC
array after sorting: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
correct: PASS
1411 instructions in 1575 cycles, 120 stalls, 439 branches
```

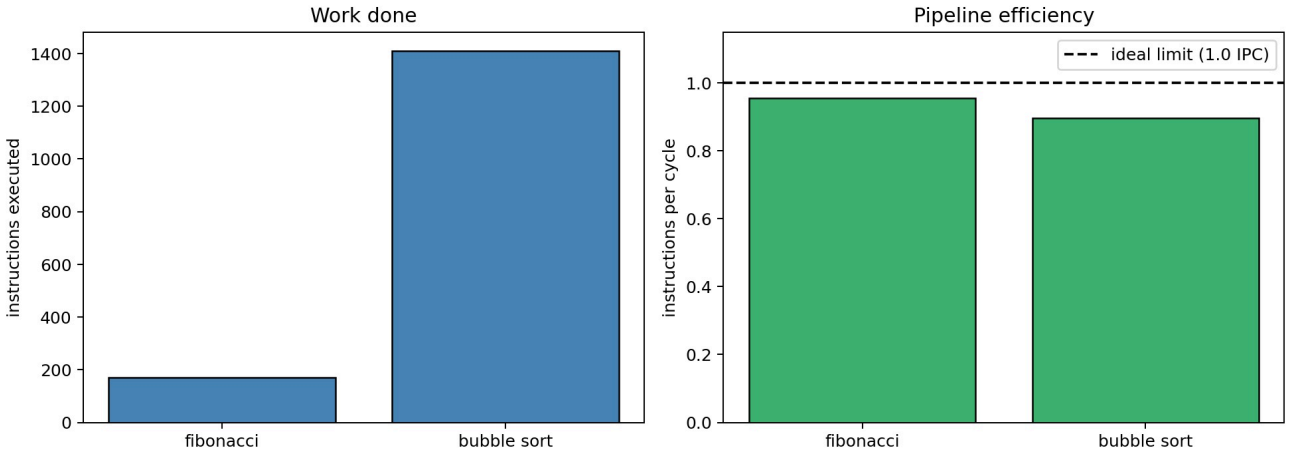


Figure 6 - Graph drawn by neurocore_demo.ipynb on the board

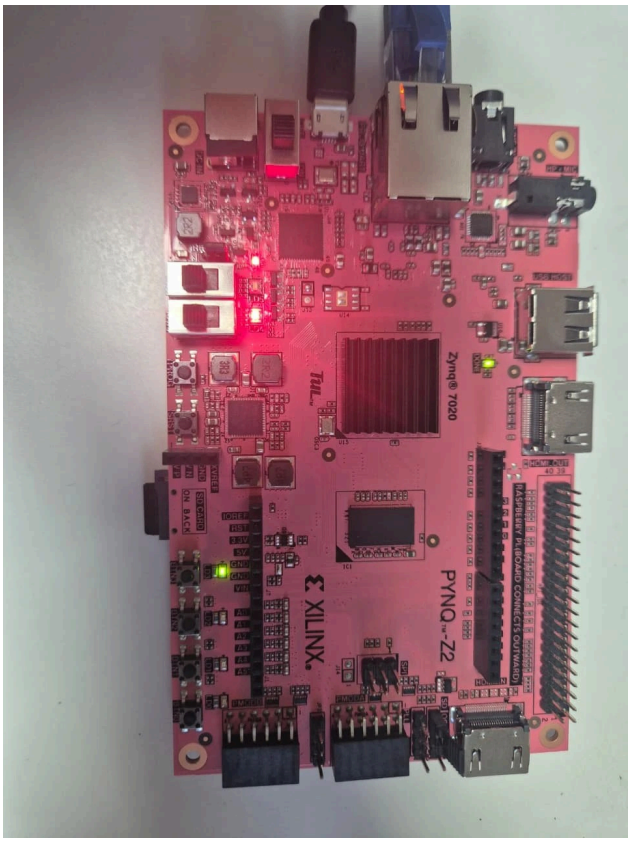


Figure 7 - PYNQ-Z2 board setup: micro-USB (power and JTAG/UART), Ethernet to the PC, PYNQ Linux on the SD card

All three programs produced the correct results, and the Fibonacci program took exactly the same number of cycles on the board as in simulation, as expected for a synchronous design. The pipeline completes close to one instruction per clock. The few lost cycles come from filling the pipeline at the start, from the two instructions flushed whenever a branch is mispredicted (for example the last iteration of each loop), and in the bubble sort from load-use stalls, where an instruction needs the value that the previous load has not yet returned.

5.3 A bug found on the board: a store after EBREAK

While testing on the board, the Fibonacci program was run straight after the sensor program. F15 (data address 0x13C) then read back as 1 instead of 610. The cause: loading Fibonacci (17 words) overwrites only the start of the instruction memory, so the word after its EBREAK was still an instruction left over from the sensor program, *sw x11, 0(x10)* (0x00B52023). The core fetches past EBREAK, so this store was one stage behind EBREAK in the pipeline. Fibonacci does not use x10 and x11, and the register file

is not cleared by a reset, so they still held the sensor program's last values: x10 = 0x13C (the address of its last alarm flag) and x11 = 1. The store wrote 1 to 0x13C in the same clock that EBREAK reached write-back, just before the core clock stopped.

Fix (riscv_core.sv): the data memory write enable is blocked while EBREAK is in the write-back stage.

```
wire ebreak_in_wb = mem_wb_out.valid & mem_wb_out.is_system;
...
.mem_wr_en (ex_mem_out.mem_wr_en & ~ebreak_in_wb),
```

A fourth test was added to the testbench that repeats this sequence (sensor program, then Fibonacci) and checks that F15 is 610. It fails with the old RTL (F15 = 1) and passes with the fix (section 4). The board results above were taken with the bitstream built before the fix; in the order used there (Fibonacci, sort, sensor) they were not affected. The bitstream was then rebuilt with the fix (2026-09-19, timing met, numbers in section 5.1); the fix itself is checked in simulation, and the new bitstream has not yet been run on the board.

6. Applications

- **Embedded and IoT controllers:** a small soft processor next to custom hardware, running control code such as the sensor alarm program on the edge instead of sending raw data away.
- **Customisable processors:** because RISC-V is open, new instructions or accelerators can be added for one application.
- **Teaching and research:** the counters make pipeline effects (stalls, mispredictions) visible and measurable.
- **Management cores:** a small CPU inside a larger FPGA design to handle configuration and housekeeping.

7. Conclusion

A five-stage pipelined RV32I processor was designed, simulated and implemented on the PYNQ-Z2. It meets timing at 50 MHz, runs programs loaded by the ARM processor, and reached an IPC close to 1 on the board with correct results for all three test programs. Testing on the board also found a bug, a store that could still complete after EBREAK; it was fixed in the RTL and a regression test was added (section 5.3).

Learning outcomes: the RV32I instruction formats and how to decode them; building a pipeline with pipeline registers; data hazards and how forwarding and stalling solve them; control hazards and how a branch target buffer with 2-bit counters reduces their cost; measuring performance with hardware counters; and sharing memories between a soft processor and the ARM over AXI.

Future scope: add the M extension (multiply and divide); use block RAM for larger memories; add interrupts and a UART so the core can talk on its own; add a misprediction counter to measure the predictor's accuracy directly; and run C programs compiled with the RISC-V GCC toolchain.

Experiment 5: Hardware Q-Learning Agent

FPGA Build Challenge 2026 – Advanced level – AMD PYNQ-Z2 (Zynq-7000 XC7Z020)

1. Objective

Reinforcement learning (RL) trains an agent to make decisions by interacting with an environment and receiving rewards. The objective of this experiment is to build an entire Q-learning agent in the FPGA fabric: the maze environment, the Q-table, the Bellman update arithmetic, the exploration strategy and the training state machine all run in hardware. The ARM processor only sets the hyperparameters and reads the results over AXI4-Lite. The agent learns to solve an 8×8 maze from scratch, with no prior knowledge of the layout, and finds the shortest path in under 5 ms of wall-clock time.

2. Background: Q-Learning

Q-learning is a model-free, off-policy RL algorithm. It maintains a table Q(s, a) of expected future reward for each state–action pair. After every step, one entry is updated with the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [\ \text{reward} + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)\ ]$$

where α is the learning rate (how fast the agent updates its beliefs), γ is the discount factor (how much it cares about future rewards), and s' is the state reached after taking action a. The agent follows an ε-greedy policy: with probability ε it picks a random action (exploration), otherwise the best known action (exploitation). ε starts high and decays after each episode, so the agent gradually shifts from exploring to exploiting.

Parameter	Value	Fixed-point representation
α (learning rate)	0.10	0x1A (0.8 unsigned: 26/256 ≈ 0.1016)
γ (discount factor)	0.95	0xF3 (0.8 unsigned: 243/256 ≈ 0.9492)
ε (initial exploration)	0.90	0xE6 (0.8 unsigned: 230/256 ≈ 0.8984)
Rewards	+100 goal, −10 wall, −1 step	Signed 8.8 fixed point (16 bits)
Episodes	2000	32-bit unsigned
Max steps per episode	256	16-bit unsigned

3. Block Diagram

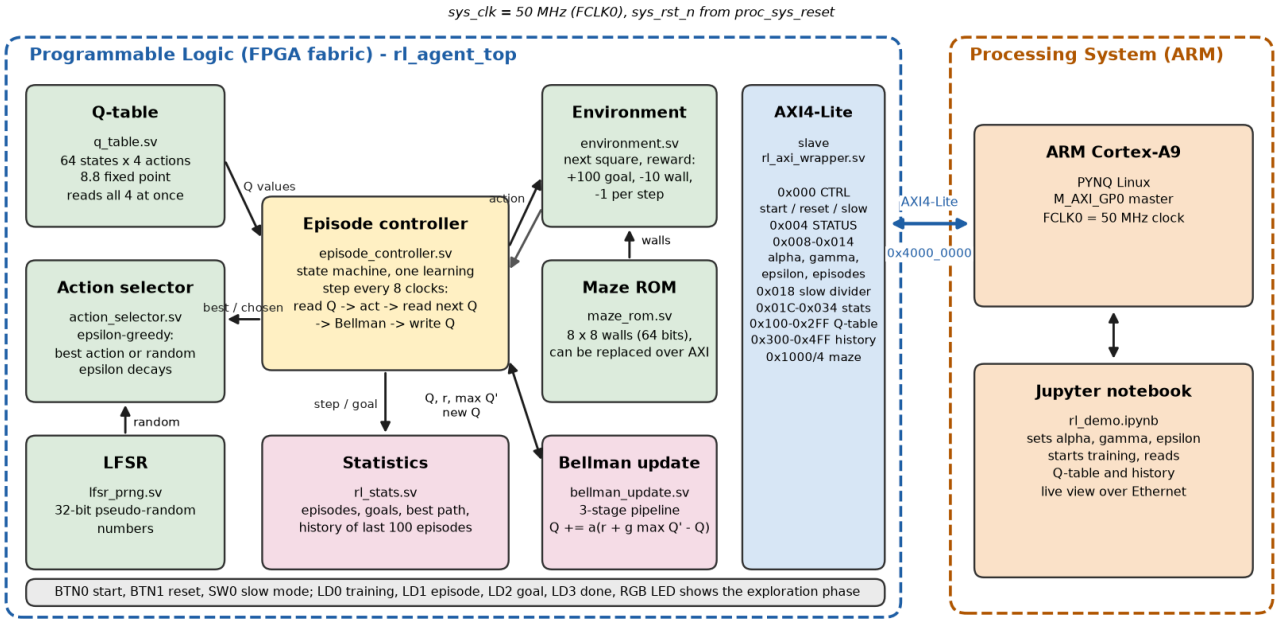


Figure 1 – System block diagram

The ARM processor writes hyperparameters (and optionally a new maze layout) into the AXI registers, triggers training, and reads the results. All learning computation happens in the fabric at 50 MHz; the ARM only observes.

4. RTL Design

4.1 Module hierarchy

```
system_wrapper                   (Vivado block design, FCLK0 = 50 MHz)
 +-- rl_agent_top_0             rl_agent_top.sv  (buttons, LEDs, RGB indicator)
      |-- u_maze_rom            maze_rom.sv
      |-- u_lfsr_prng           lfsr_prng.sv
      |-- u_q_table             q_table.sv
      |-- u_action_selector     action_selector.sv
      |-- u_environment         environment.sv
      |-- u_bellman_update      bellman_update.sv  (3-stage pipeline)
      |-- u_rl_stats            rl_stats.sv
      |-- u_episode_controller  episode_controller.sv  (11-state FSM)
      +-- u_rl_axi_wrapper      rl_axi_wrapper.sv
```

4.2 Modules

Module (file)	What it does
maze_rom	Stores the 8×8 maze layout as a 64-bit register (one bit per cell: 0 = open, 1 = wall). The ARM can write a new maze layout at runtime through the AXI registers.
lfsr_prng	A 32-bit linear feedback shift register (seed 0x5A8E1247). It gives a 16-bit random number for the explore/exploit decision and a 2-bit random action.
q_table	64 states × 4 actions, each a signed 8.8 fixed-point value. Supports single-cycle read of all four actions for one state, single-action write, and a 64-cycle sweep to clear the entire table on reset.
action_selector	Compares the four Q-values to find the best action and its Q-value (argmax). Compares a random 16-bit number against the current epsilon threshold to decide explore vs. exploit. After each episode epsilon is multiplied by 0.995 until it reaches the minimum of 0.05.
environment	Pure combinational: given the current position and action, checks the maze layout for walls and boundaries, computes the next position and the reward (+100 goal, −10 wall, −1 step).
bellman_update	3-stage pipeline implementing $Q_{new} = Q_{old} + \alpha \cdot (\text{reward} + \gamma \cdot \max Q(s') - Q_{old})$. Uses signed fixed-point arithmetic with saturation clamping to [−128.0, +127.996].
rl_stats	Counts episodes, steps, total reward, best path length, and total goals reached. Maintains a circular buffer of the last 100 episodes (goal flag + step count), readable from AXI.
episode_controller	11-state FSM that runs one learning step every 8 clock cycles: read Q-values, select action, query environment, read next-state Q-values, run Bellman update, write updated Q, check if the episode is done (goal, wall or 256 steps). In slow mode it waits between steps so the agent can be watched live.
rl_axi_wrapper	AXI4-Lite slave: control (start, reset, slow mode), status, hyperparameters, statistics, Q-table readout (128 words, 2 values each), episode history buffer (100 entries), and maze load registers. Total address space: 0x2000.
rl_agent_top	Top-level wrapper: instantiates all modules, synchronises the push buttons, drives the four LEDs and the RGB LED, and combines AXI and button controls.

4.3 Register map (base address 0x4000_0000)

Offset	Name	Access	Meaning
0x000	CTRL	W	bit 0 start, bit 1 reset, bit 2 slow mode
0x004	STATUS	R	bit 0 training, bit 1 finished, bits [31:16] episode number
0x008	ALPHA	R/W	learning rate (0.8 unsigned fixed-point, default 0x1A)
0x00C	GAMMA	R/W	discount factor (0.8 unsigned fixed-point, default 0xF3)
0x010	EPSILON	R/W	initial exploration rate (0.8 unsigned fixed-point, default 0xE6)
0x014	MAX_EPISODES	R/W	number of episodes to train (default 2000)
0x018	SLOW_DIVIDER	R/W	clock cycles per step in slow mode (default 2,500,000 = 50 ms)
0x01C–0x034	STATS	R	episode, steps, reward, position, best path, epsilon, total goals
0x100–0x2FF	Q_TABLE	R	128 words: 2 signed 8.8 Q-values per word (UP/DOWN then LEFT/RIGHT)
0x300–0x4FF	HISTORY	R	100 entries: bit 31 = goal reached, bits [23:0] = step count
0x1000	MAZE_LO	R/W	lower 32 bits of the 64-bit maze layout
0x1004	MAZE_HI	R/W	upper 32 bits; writing this register loads the new maze

4.4 Board LEDs and controls

I/O	Function
LD0	Training is running
LD1	Toggles at the end of every episode
LD2	Flashes for ~100 ms when the agent reaches the goal
LD3	Training has finished
RGB LD4	Red (exploring, $\varepsilon > 0.60$), yellow = red + green (balanced, $0.20 < \varepsilon \leq 0.60$), green (exploiting, $\varepsilon \leq 0.20$). See the note below about the pins.
BTN0	Start training
BTN1	Reset everything
SW0	Slow mode (one step every 50 ms for the live maze view)

Note on the RGB LED pins. The first constraints file mapped rgb_led0 to pins M15, G14 and L15. On the PYNQ-Z2, M15 is the red and G14 the blue of LD5, so with that bitstream the colour appeared on LD5 as red → magenta → blue. The pins were corrected to N15, G17 and L15 (LD4 red, green, blue) and the bitstream was rebuilt on 2026-09-19 (timing met, numbers in section 6.1), so LD4 now shows red → yellow → green. The board results in section 6.2 were taken with the earlier bitstream; the change only moves the LED pins and does not touch the learning logic.

4.5 RTL schematics

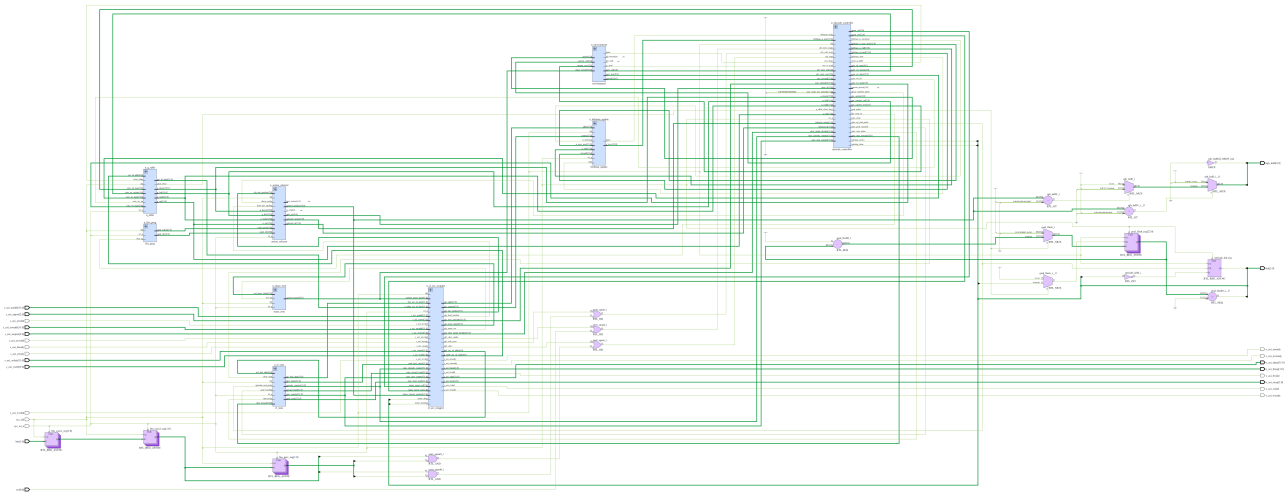


Figure 2 – Elaborated RTL schematic of rl_agent_top (Vivado)

5. Simulation Results

The design was simulated with Icarus Verilog 12 using the testbench *rl_agent_tb.sv*, which acts as the ARM processor over AXI4-Lite and trains the agent for the full 2000 episodes on the built-in maze (details in Simulation/simulation_report.pdf). All checks passed:

#	Check	Result
1	Default settings (alpha 0x1A, gamma 0xF3, 2000 episodes)	**PASS**
2	Training runs all 2000 episodes and reaches the goal	**PASS** – 1335 goals
3	Best path is the 14-step shortest path	**PASS** – 14 steps
4	Move into the goal learned, Q((7,6), RIGHT) > 80	**PASS** – 96.37

```
finished: 2000 episodes, 1335 goals, best path 14 steps
[PASS] the agent found the shortest path through the maze
Q((7,6), RIGHT) = 96.37
[PASS] the value of stepping into the goal was learned
history entry 0: goal 1, 14 steps
PASS: the hardware agent learned to solve the maze
```

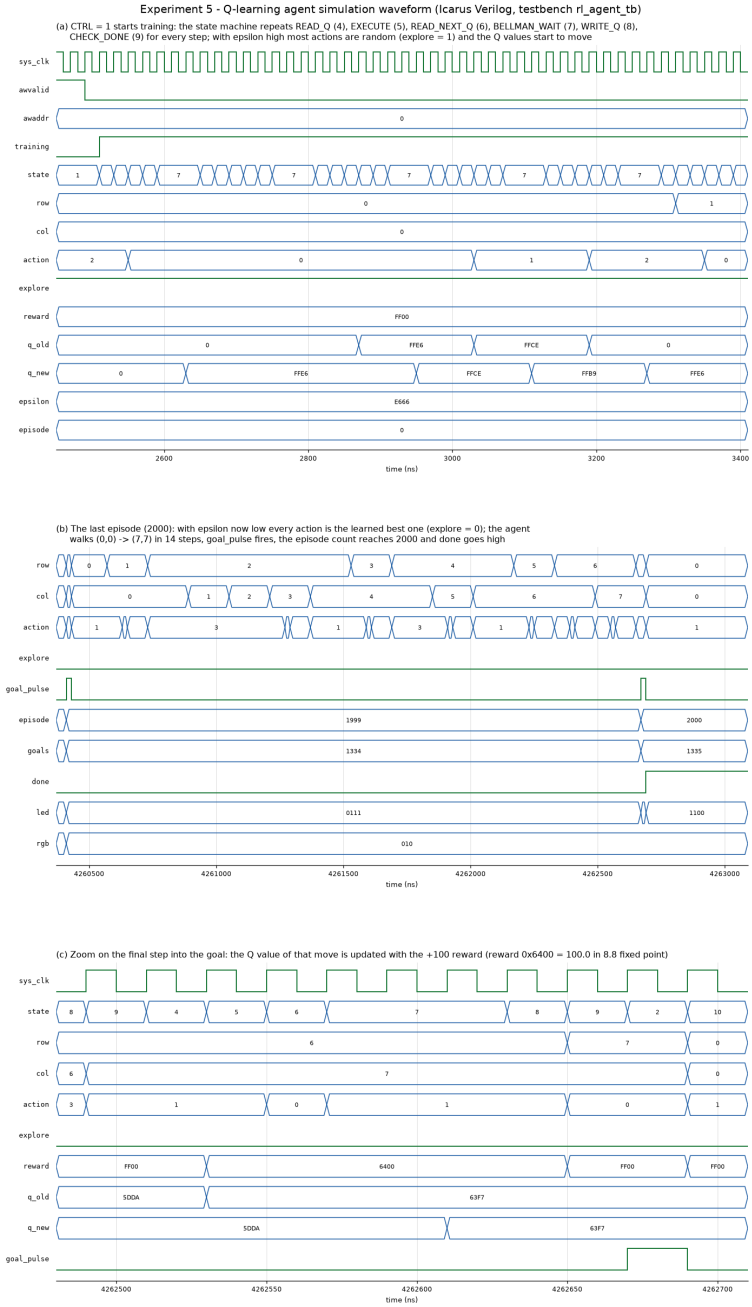


Figure 3 – Simulation waveform: start of training, the last episode and the final step into the goal

6. Hardware Implementation

6.1 Build

The design was built with Vivado 2026.1 for the xc7z020clg400-1 device using build.tcl, with the fabric clock set to 50 MHz via the processing system's FCLK_CLK0. The constraints file (pynq_z2.xdc) assigns the four LEDs, the RGB LED, the two buttons and the switch. The Q-table (64 states × 4 actions × 16 bits = 4,096 bits) is written one entry at a time but all four values of a state are read at once, so Vivado builds it from flip-flops and multiplexers instead of block RAM. That is why the Q-table uses most of the registers and LUTs, and no block RAM is used.

Resource	Used	Available	Utilisation
Slice LUTs	6,576	53,200	12.36 %
of which LUT as memory (shift registers)	60	17,400	0.34 %
Slice registers (flip-flops)	7,876	106,400	7.40 %
Block RAM tiles	0	140	0.00 %
DSP48 blocks	3	220	1.36 %
Bonded I/O pins	10	125	8.00 %
Global clock buffers	1	32	3.13 %

Timing (50 MHz clock)	Value	Status
Worst setup slack (WNS)	+2.473 ns	**met**
Worst hold slack (WHS)	+0.049 ns	**met**
Timing endpoints	16,603	

With a WNS of +2.473 ns, the design has 2.5 ns of positive slack and could run at approximately 57 MHz if needed.

6.2 Test on the board

The steps of the notebook *rl_demo.ipynb* (Jupyter folder) were run on the PYNQ-Z2 with PYNQ (tools/execute_all_on_pynq.py runs them on the board over SSH and copies the log and graphs back; tools/embed_nb_outputs.py puts them into the notebook). The run loads the bitstream, trains the agent for 2000 episodes with the default settings, reads the statistics, the Q-table and the episode history over AXI, runs one episode in slow mode for the live view, and then loads three new mazes over AXI and trains on each of them. The log of the run and the graphs it drew are in Jupyter/output. The numbers below are taken from that log and compared with the simulation by tools/p5_rl_results.py.

Check	Detail	Result
Training ran all episodes	2000 episodes, 1335 reached the goal	**PASS**
Shortest path found	14 steps (shortest possible 14)	**PASS**
Board result equals simulation	1335 goals on the board, 1335 in simulation	**PASS**
Live run reaches the goal	slow-mode episode found the goal in 14 steps	**PASS**
New mazes loaded over AXI	Corridor Detour: 14, Center Barrier: 14, Zig-Zag Staircase: 14	**PASS**

The board gives exactly the same result as the simulation (1335 goals, best path 14). This is expected: the LFSR starts from the same seed, so the hardware makes the same random choices in the same order.

Board log (Jupyter/output/terminal_output.txt)

```
[*] Flashing FPGA bitstream for Experiment 5 (RL Agent)...
[+] FPGA Flashed! DONE LED active.
Overlay loaded, agent at 0x40000000
episodes      : 2000
goals reached : 1335
best path     : 14 steps (shortest possible is 14)
final epsilon : 0.050
measured 4.9 ms for 2000 episodes on hardware
[Ethernet Visualizer] Streaming live learning dynamics over Ethernet (8.0s)...
[Ethernet Visualizer] GOAL REACHED! Target found in 14 steps! Total goals: 1336
[+] Saved live visualizer frame: /tmp/pynq_runs/exp5_graph_2_live_visualizer_frame.jpg
Maze 'Corridor Detour': best path = 14 steps
Maze 'Center Barrier': best path = 14 steps
Maze 'Zig-Zag Staircase': best path = 14 steps
```



Figure 4 – Results of rl_demo.ipynb on the PYNQ-Z2: learned policy, three new mazes and the live view

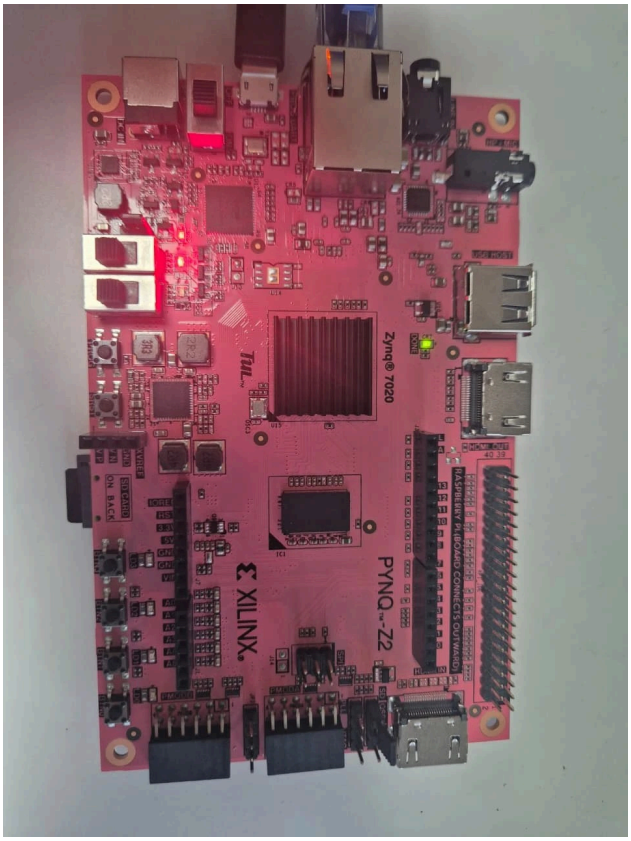


Figure 5 – PYNQ-Z2 board setup: micro-USB (power and JTAG/UART), Ethernet to the PC, PYNQ Linux on the SD card

6.3 Learning progress

The simulation transcript prints the goal count every few episodes. It shows the usual three phases of ε-greedy Q-learning (the board run gives the same totals):

Phase	Episodes	Goals reached	What happens
Exploring	0 – ~360	0	ε is high, almost every action is random and most episodes end at a wall
Transition	~360 – ~500	60 by episode 496	the goal is found and its value spreads back through the Q-table
Exploiting	~500 – 2000	1335 by episode 2000	about 80 % of episodes reach the goal; ε settles at 0.05

In the last episode (Figure 3b) the agent follows the learned policy and walks the 14-step shortest path. The three extra mazes (Corridor Detour, Center Barrier and Zig-Zag Staircase) were loaded over AXI without a new bitstream, and the agent found the 14-step shortest path in each of them.

6.4 Training speed

At 50 MHz one learning step takes 8 clock cycles (160 ns). An episode ends at the goal, at a wall or after 256 steps, so the early exploring episodes are short. The whole 2000-episode training took 4.4 ms of simulated time, and 4.9 ms was measured on the board from Python (this includes the time Python spends polling the status register). A Python Q-learning loop on the ARM core needs several microseconds for each step, so the hardware is much faster.

7. Applications

- **Autonomous navigation:** a hardware RL agent at the edge of an embedded system learns to navigate physical environments (e.g. a robot in a warehouse grid) in real time, without waiting for a cloud server.
- **Adaptive control:** the Q-table can be mapped to sensor readings and actuator commands, so the agent learns to optimise power management, thermal regulation, or network routing directly in hardware.
- **Hardware-accelerated AI research:** running RL episodes in the FPGA fabric is much faster than software, so many hyperparameter settings can be tried quickly.
- **Teaching and demonstration:** the LEDs and RGB indicator make the learning process visible, and the live maze view in slow mode lets students watch the agent explore and exploit in real time.

8. Conclusion

A complete Q-learning reinforcement learning agent was designed, simulated and implemented in the FPGA fabric of the PYNQ-Z2. The agent learns to solve an 8×8 maze from scratch with no prior knowledge, finds the optimal 14-step path, and trains 2000 episodes in under 5 ms. All simulation and board checks passed, the board result is identical to the simulation, and three new mazes loaded at run time were also solved. The only issue found, the RGB LED pin mapping, was fixed in a rebuilt bitstream (section 4.4).

Learning outcomes: the Bellman equation and how to implement it in fixed-point arithmetic; epsilon-greedy exploration and how exponential decay balances exploration with exploitation; designing a multi-state FSM that coordinates pipelined arithmetic with memory reads and writes; building a complete AXI4-Lite register file with read-only statistics, read-write configuration, and memory-mapped table readout; and using the PYNQ framework to interact with custom hardware from Python.

Future scope: implement Deep Q-Networks (DQN) with a small neural network in the fabric instead of the flat Q-table; add SARSA or other on-policy algorithms for comparison; support larger mazes with block RAM storage; add HDMI output for real-time maze visualisation without the notebook; and implement multi-agent RL with several agents sharing the same maze.